

DISEÑO DE INTERFACES WEB

U.D.2 USO DE ESTILOS

1. HTML. HISTORIA

- **1989** Inicio desarrollo de la Web
- **1991** Lanzamiento de la Web. **Sir Tim Berners-Lee** escribió el documento «*HTML Tags*» que describía los 20 elementos que deberían ser utilizados para escribir documentos web.



1. HTML. HISTORIA

- **1991** Lanzamiento de la Web
- **1992** Lanzamiento de HTML
- **1994** Creación de W3C
- **1995** HTML 2.0
- **1998** HTML 4
- **1999** HTML 4.01 -XHTML
- **2004** WHATWG
- **2008** HTML 5
- **2014** Recomendación HTML 5 (W3C)
- **2016** HTML 5.1
- **2017** HTML 5.2



1. HTML. HISTORIA

- En 1999 tras HTML 4.01, el W3C decidió suspender el desarrollo de HTML para trabajar con un nuevo lenguaje XHTML basado en XML, es un lenguaje complejo para compartir información que tuvo la respuesta en JSON(más limpio). Como respuesta en 2004 se creó el grupo WHATWG con ingenieros de Apple, Mozilla, Opera, Google, etc que crearon HTML5. Fue tan revolucionario que W3C al final suspendió XHTML.



1. HTML. HISTORIA

Objetivos del HTML5

- Asegurar la compatibilidad con lo que ya existe.
- La especificación describe con detalle lo que deben hacer los navegadores, lo que estos deben mostrar. Se regula por fin la gestión de los errores, si no es correcto, ¿qué deberá mostrar el navegador?
- Gestionar correctamente la interoperabilidad, la compatibilidad de los navegadores

1. HTML. HISTORIA

Especificaciones(estándares) de la W3C



Etapas de la especificación:

- **Working draft (WD)**. Es el borrador. Ej. HTML 5.3
- **Candidate recommendation(CR)**. En desarrollo para pruebas.
- **Proposed recommendation(PR)**. Para pruebas finales.
- **Recommendation(REC)**. Listo para usar.

1. HTML. HISTORIA

Navegadores

- **1993** - Mosaic
- **1994** - Netscape
- **1995** - Internet Explorer
- **1996** - Opera
- **2003** - Safari
- **2004** - Mozilla Firefox
- **2008** - Google Chrome
- **2015** - Microsoft Edge



1. HTML. HISTORIA

En la actualidad, el navegador parece el Sistema Operativo utilizando HTML5 , CSS y JavaScript.

- **PWA** (Las aplicaciones web progresivas están a medio camino entre las web y las nativas: son básicamente páginas web, pero mediante el uso de Service Workers y otras tecnologías se comportan más como aplicaciones normales que como aplicaciones web, pueden seguir ejecutándose en segundo plano sin tener que vivir dentro del navegador. Son una evolución natural de las aplicaciones web que difuminan la barrera entre la web y las aplicaciones, pudiendo realizar tareas que generalmente solo las aplicaciones nativas podían llevar a cabo. Algunos ejemplos son las notificaciones, el funcionamiento sin conexión a Internet o la posibilidad de probar una versión más ligera antes de bajarte una aplicación nativa de verdad con significado)
- **Electron**
- **Ionic**
- **React Native**
- **Web Components** (seran muy utilizados en futuro próximo)

1. HTML.

HTML5

- **Web semántica** (con significado)
- **Local Storage** (guardar datos en local)
- **Acceso al dispositivo** (acceder a cámara, micro, etc desde el navegador)
- **Web sockets** (comunicación bidireccional abierta en tiempo real con servidor sin petición ni F5)
- **Multimedia** (Audio y Video nativo (Antes todo era Flash, ahora todo lo hace el navegador))
- **Gráficos** (**SVG** (gráficos vectoriales bidimensionales), **Canvas**(elemento para gráficos y animaciones) , **WebGL**(API JavaScript para renderización de gráficos en 3D))
- **Web workers** (Un 2 proceso para atender trabajos en JS simultáneos sin ocupar el hilo principal)
- **CSS3** (lenguaje de diseño gráfico para definir y crear la presentación de un documento estructurado escrito en un lenguaje de marcado)

1. HTML.

¿Diferencias de HTML5 respecto a XHTML?

- No es necesario cerrar los elementos vacíos.

```
<input type="text">
```

- Algunos elementos (p.e. <p>) no es necesario cerrarlos, pero los cerraremos para tener un código más limpio.
- El uso indistinto de mayúsculas y minúsculas no afecta a la validación de las páginas. Pero por claridad es aconsejable utilizar siempre minúsculas.

1. HTML.

¿Diferencias de HTML5 respecto a XHTML?

- Tampoco es obligatorio poner comillas en los atributos. Hay ocasiones en cambio en que si será necesario, por ejemplo cuando declaramos múltiples clases separadas por espacios.

```
<article class="column wrapper">
```

- Por consistencia es aconsejable poner siempre las comillas.
- No es necesario poner el valor en los atributos booleanos.

```
<input type="checkbox" checked>
```

1. HTML.

PLANTILLA de HTML5 .

<https://www.htmlquick.com>

Elementos fundamentales que debe tener cualquier documento HTML.

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <meta name="description" content="Poner una descripción fenomenal de la página">
  <title>Titulo de la página web</title>
  <link rel="stylesheet" href="estilos.css">
  <link rel="shortcut icon" href="icono.png">
</head>
<body>
  <h1>Estructura básica típica</h1>

  <script> src="scripts.js"</script>
</body>
</html>
```

1. HTML.

DOCTYPE

```
<!DOCTYPE html>
```

El **Document Type Declaration** (Doctype) su finalidad indicarle al navegador de qué tipo de documento se trata. Se indica de la forma:

html

No tiene cambios muy significativos.

```
<html lang=es>
```

El atributo xmlns, requerido en XHTML ya no es necesario.

El atributo lang tampoco es obligatorio.

1. HTML.

head

La primera línea dentro de head(se describen los metadatos) es la que define la codificación de caracteres:

```
<meta charset="utf-8">
```

Esta línea también se ha simplificado.

Para evitar problemas esta línea tiene que estar en los primeros 512 caracteres del documento y antes de cualquier contenido (p.e. <title>)

1. HTML.

head

Otra característica dentro del head es que al enlazar con las hojas de estilo ya no es necesario el atributo type="text/css"

```
<link rel="stylesheet href="css/styles.css">
```

1. HTML.

etiquetas

Etiquetas semánticas: *su objetivo es describir qué es el contenido que contienen, en lugar de cómo se debe formatear.* Ej. main, header, section, article, aside, footer y nav.

Ventajas: los motores de búsqueda comprenderán mucho mejor el contenido de tu página

1. HTML.

Ejemplo Etiquetas semánticas:

```
<body>
  <header></header>
  <nav></nav>
  <article>
    <h1>Titulo</h1>
    <section>
      <h2>Subtitulo</h2>
    </section>
  </article>

  <aside></aside>
  <footer></footer>
</body>
```

1. HTML.

<header>

Representa un encabezado para ayudas introductorias o de navegación. Puede contener algunos elementos de encabezado, así como también un logo, menú de navegación, un cuadro de búsqueda, un nombre de autor y otros componentes.

Su uso no está limitado a la cabecera de la página, también se puede utilizar en los encabezados de las secciones.

1. HTML.

<nav>

Representa una sección de una página que enlaza con otras páginas o a partes dentro de la propia página: **una sección con enlaces de navegación.**

Este elemento debe reservarse para enlaces de navegación importantes.

Por ejemplo: no es aconsejado su uso para una breve lista de enlaces en el pie de página

1. HTML. HISTORIA

<nav>

Una página puede tener más de un elemento <nav>.

Por ejemplo:

- Barra de navegación superior para todo el sitio
- Barra de navegación lateral para secciones de la página actual.

1. HTML.

<main>

Representa el contenido principal del `<body>` de un documento o aplicación. El elemento **main** es un contenedor para el tema central del documento o sección. El tema central puede ser interpretado como todo lo que queda después de eliminar headers (header), footers (footer), nav sections (nav), related links y otros como elementos secundarios.

No debe haber más de un elemento <main> en un documento, y este no debe ser descendiente de un elemento `<article>`, `<aside>`, `<footer>`, `<header>`, o `<nav>`.

1. HTML.

<article>

- Este elemento es un contenedor de bloques de contenido que se consideran independientes del documento o sitio web y, por lo tanto, se puede ver, reutilizar y distribuir por separado. Por lo general, puede encontrar este elemento envolviendo artículos, entradas de blog o publicaciones en foros.
- Cuando dos o más de estos elementos están anidados, se supone que el artículo interno mantiene relación con el artículo que actúa como padre.

1. HTML.

<section>

Para definir contenido genérico y dividirlo en diferentes secciones.
La sección divide la página en bloques por tema..

Antes, los encabezados se usaban solos para definir secciones, en HTML5, los elementos de sección son responsables de la división del contenido, mientras que los encabezados(h1...h6) proporcionan un título para cada sección. Cuando se utiliza este enfoque, el encabezado (generalmente un h2) precede a todo el contenido de la sección y proporciona una descripción concisa al respecto.

1. HTML.

<section>

```
<section>
```

```
  <h1>Biografia de Mozart </h1>
```

```
  <section>
```

```
    <h2>Vida</h2>
```

```
    <p>Wolfgang Amadeus Mozart nació el 27 de enero de 1756 en Salzburgo, en la actual Austria. Fue el último hijo de Leopold Mozart, músico al servicio del príncipe arzobispo de Salzburgo. Falleció en Viena a la edad de 35 años</p>
```

```
  </section>
```

```
  <section>
```

```
    <h2>Producción musical</h2>
```

```
    <p>Mozart aparece hoy como uno de los más grandes genios musicales de la historia. Fue excelente fortepianista, organista, violinista y director, destacaba por sus improvisaciones, que solía realizar en sus conciertos...</p>
```

```
  </section>
```

```
</section>
```

1. HTML.

<address>

Aporta información de contacto para su <article> más cercano o ancestro <body>; en el último caso lo aplica a todo el documento.

Este elemento debe contener solamente información de contacto.

<address>

Puedes contactar con el autor en

www.algundominio.com.

Si ves errores, por favor contacta con webmaster.

También puedes visitarnos:

IES Los Sauces

Avda. Federico Silva, 48

49600- Benavente

Spain

</address>

1. HTML.

<hgroup>

Permite agrupar etiquetas h que mejoran la búsqueda SEO además de poder repetir varias de ellas en distintos grupos.

Ej: <hgroup>

```
    <h1>Main title</h1>
```

```
    <h2>Secondary title</h2>
```

```
</hgroup>
```

```
<hgroup>
```

```
    <h1>Main article</h1>
```

```
    <h2>Secondary article</h2>
```

```
</hgroup>
```

1. HTML.

<aside>

- Representa una parte de la página con contenido que está indirectamente o no relacionado con el contenido principal del documento.
- Nunca se debe utilizar para rodear secciones que son parte del contenido principal. Ej: Periodico con publicidad al costado.

1. HTML.

<aside>

Este elemento se puede utilizar para:

- Una explicación al margen como una definición de glosario
- Elementos relacionados indirectamente:
 - Publicidad
 - Biografía del autor.
 - En aplicaciones web, la información de perfil o enlaces a blogs relacionados.

1. HTML.

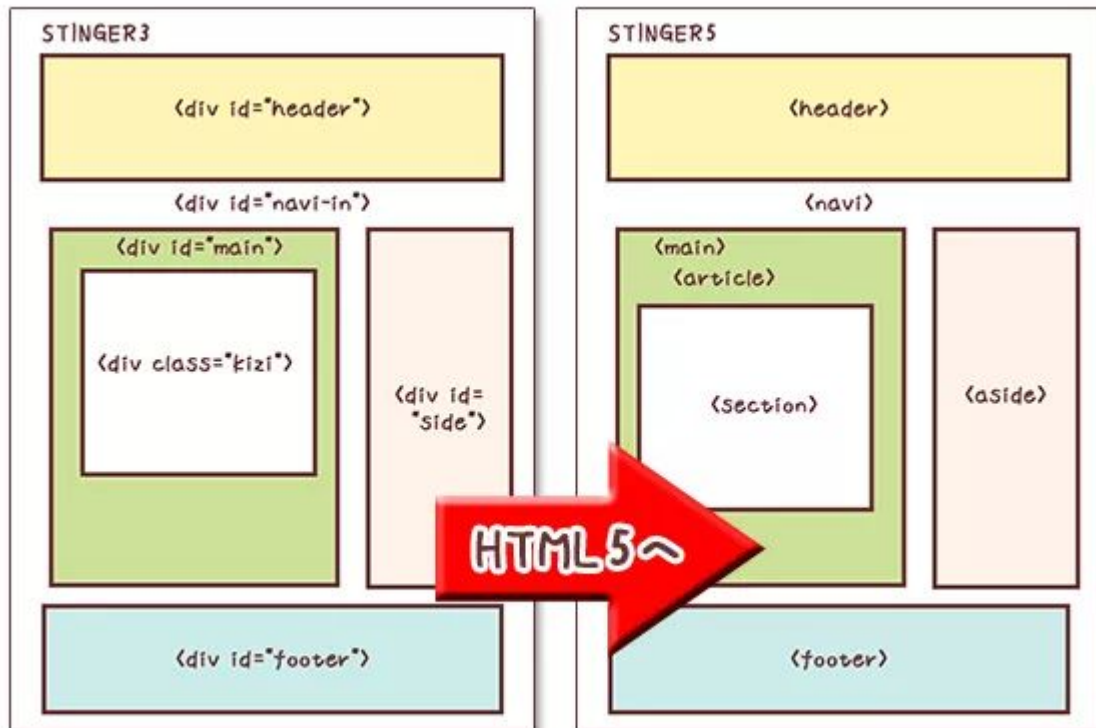
etiquetas <footer>

- Representa un pie de página para la sección de contenido que es su ascendiente más próximo.
- Cuando en la definición se habla de sección puede ser el documento entero pero también un elemento **section**, **article** o **aside**.

Esta etiqueta puede contener:

- Información de copyright
- Lista de enlaces relacionados
- Información sobre el autor, notas al pie, etc.

1. HTML.



1. HTML.

<figure> y <figcaption>

Representa a una pieza de documento autocontenido (tiene cierto grado de independencia del flujo del documento) que se utiliza habitualmente para encerrar imágenes, gráficos, tablas de referencia, videos, poemas, fragmentos de código, etc. Puede ser acompañada por una leyenda o título (elemento `figcaption`).

```
<h1>Análisis de uso de los navegadores </h1>
<figure>
  
  <figcaption>Gráfico A: Uso de navegadores </figcaption>
</figure>
```

1. HTML.

<picture>

HTML5 introduce el elemento `<picture>` para añadir flexibilidad al recurso image. El elemento `<picture>` contiene n elementos `<source>` valorados de arriba a abajo, cada uno referenciando un recurso de imagen diferente. Con el atributo `media`, el browser puede elegir la imagen que mejor se adapta a la vista actual y o dispositivo. Es obligatorio el elemento `img`. El atributo `orientation` tiene valores `portrait` y `landscape` . También podemos especificar el tipo de imagen: jpg, png, svg, gif, etc.

Ej: `<picture>`

```
<source media="(min-width: 800px)" srcset="img_grande.jpg">
<source media="(min-width: 640px)" srcset="img_mediana.jpg">
<source media="(max-width: 639px)" srcset="img_small.jpg">
  
</picture>
```

hasta que ha llegado picture, se intentaba con img, ej:

```

```

1. HTML.

<picture>

Ej: <picture>

```
<source media="(orientation: portrait) and (max-width: 400px)"
srcset="img-portrait400.jpg">
<source media="(orientation: portrait) " srcset="img_portrait800.jpg">
/*landscape*/

<source media="(min-width: 800px)" srcset="img_grande.jpg">
<source media="(min-width: 640px)" srcset="img_mediana.jpg">
<source media="(max-width: 639px)" srcset="img_small.jpg">


</picture>
```

1. HTML.

<picture>

```
Ej: <picture>                                /* por tipo de imagen: */  
    <source type="image/webp" srcset="imagen.webp">  
      
</picture>
```

```
<picture>    /* por resolución de pantalla, baja: 1x, alta: 2x, muy alta: 3x  
siempre mostrará la más alta que pueda reproducir: */  
    <source srcset="imagen-small.jpg , imagen-big.jpg 2x " srcset="imagen.webp">  
      
</picture>
```

1. HTML.

I also have some kittens who are visiting me these days. They're really cute. I think they like my garden! Maybe I should adopt a kitten.

<mark>

- Representa un fragmento de texto en un documento remarcado con propósito de referenciarlo, debido a su relevancia en otro contexto.
- El uso más común de esta etiqueta es en el contexto de una búsqueda, donde las palabras clave que se han buscado se muestran resaltadas.
- En este contexto la etiqueta debería añadirse mediante código en el lado del servidor o bien mediante JavaScript una vez que haya cargado la página.

1. HTML.

<progress> y <meter>

- **progress** se utiliza para visualizar el progreso de una tarea. aunque básicamente aparece una barra de progreso.
- **meter** (nuevo en HTML5) representa un valor cuyo rango es conocido, significando esto que tiene unos valores mínimo y máximo conocidos. Las barras que representan el valor son de color verde por defecto, pero si el dato se encuentra entre los valores min y low o high y max serán de color amarillo..

1. HTML.

<progress>

- Un ejemplo para el uso de **progress** sería la tradicional barra indicando el progreso de una descarga.

progress: 

Atributos de <progress>

Attribute	Value	Description
<code>max</code>	<i>number</i>	Specifies how much work the task requires in total
<code>value</code>	<i>number</i>	Specifies how much of the task has been completed

```
<p> Status:  
<progress max="100" value="0">  
  <span>0</span>%  
</progress></p>
```

1. HTML.

<meter>

Los atributos que se pueden usar en esta etiqueta son:

- **value**: Valor actual del elemento, debe ser mayor que el atributo min y menor que el max.
- **min**: Valor mínimo del elemento, si se omite el valor es 0. Debe ser menor que el atributo max.
- **max**: Valor máximo del elemento, si se omite el valor es 1. Debe ser mayor que el atributo min.
- **low**: Límite superior del valor mínimo.
- **high**: Límite inferior del valor máximo.
- **optimum**: El valor más adecuado de todo el rango. Debe estar entre el valor min y max, en caso de tener definidos los atributos high y low también deberá estar entre ambos.

Las barras que representan el valor son de color verde por defecto, pero si el dato se encuentra entre los valores min y low o high y max serán de color amarillo o rojo.

Orden: `min <= low <= optimum <= high <= max`

1. HTML.

<meter> Ej:

```
<p>
  El obtuvo un
  <meter min="0" max="10" low="5" high="9"
  value="4"></meter>
  en el examen.</p>
<p>
  Utilizacion de disco:
  <meter low="750" high="1199" max="1400"
  value="1200">
  </meter> 1200 Gb
</p>
<label for="fuel">
  Nivel combustible:</label>
<meter id="fuel"
  min="0" max="100"
  low="33" high="66" optimum="60"
  value="50">
</meter> 50%
```

El obtuvo un  en el examen.

Utilizacion de disco:  1200 Gb

Nivel combustible:  50%

1. HTML.

<input type="number">

Este elemento valida automáticamente que los valores introducidos sean numéricos sin necesidad de scripts adicionales, toda la validación la realiza el propio navegador.

```
<form name="ejemplo" >  
  Introduce tu edad (min=18 max=99, campo obligatorio):  
  <input type="number" name="edad" min="18" max="99" step="1" required="required">  
  <input type="submit" value="Enviar">  
</form> |
```

Introduce tu edad (min=18 max=99,
campo obligatorio):

1. HTML.

<input type="date">

Este elemento vale para representar en nuestra web un **datepicker** o **calendario**, el formato de la fecha devuelto es 'AAAA-MM-DD'. De esta manera podemos ahorrarnos los plugins de [jQuery](#) para utilizar calendarios y/o datepickers en nuestras web.

Fecha nacimiento:

```
Fecha nacimiento: <input type="date" name="cumpleaños"
step="1" min="1996-01-01" max="2019-11-30" value="2019-01-01">
```

febrero de 2007 ▾

lu.	ma.	mi.	ju.	vi.	sá.	do.
29	30	31	1	2	3	4
5	6	7	8	9	10	11
12	13	14	15	16	17	18
19	20	21	22	23	24	25
26	27	28	1	2	3	4

1. HTML.

<keygen>

El elemento `keygen` representa un campo para la generación de llaves criptográficas. Este elemento es nuevo en HTML5 y ha sido diseñado para ayudar en la generación de llaves usadas en la construcción de certificados firmados digitalmente. La funcionalidad de este elemento es beneficiosa en sistemas de manejo de certificados basados en web.

Esta etiqueta sirve para **enviar claves en el autenticado web**. Al enviarse el formulario se crean 2 claves, una pública que se envía al servidor y que posteriormente se usará para seguir autenticándonos en el sistema, y otra privada que se almacena localmente en el navegador.

Sintaxis: `<keygen name="clave">`

```
<form action="../../../form-result.php" method="post" target="_blank">
```

```
  <p>    Generar un par de llaves pública/privada:
```

```
  <keygen name="llavepublica">
```

```
  <input type="submit" value="Generar">
```

```
  </p>
```

```
</form>
```

Generar un par de llaves pública/privada:

1. HTML.

<details>

Representa una parte del documento que está oculta, pero que puede ser expandida para mostrar más información.

Esta funcionalidad se suele realizar en JavaScript pero la introducción de esta etiqueta elimina su necesidad, aligerando el código.

Attribute	Value	Description
<code>open</code>	 open	Specifies that the details should be visible (open) to the user

► Bibliografía

▼ Bibliografía

- *PC Wordl*
- *Computer Hoy*
- *Computer World*

```
<details>
  <summary>Bibliografía</summary>
  <ul>
    <li><cite>PC Wordl</cite></li>
    <li><cite>Computer Hoy</cite></li>
    <li><cite>Computer World</cite></li>
  </ul>
</details>
```

1. HTML.

<details>

Necesita la etiqueta `<summary>` para el título, sino el navegador define uno por defecto. La etiqueta `<summary>` debe ser el primer hijo de `<details>`. Lo que esta después de `<summary>` será el contenido de `<details>`

```
<details>  
  <summary>Titulo</summary>  
  <p>Lorem ipsum dolor sit amet consectetur adipisicing elit. Beatae  
    velit praesentium eligendi nobis adipisci ipsum aliquid accusamus  
    reiciendis, consequuntur corrupti ea doloremque quas aut qui. Libero,  
    nobis? Enim, debitis expedita.</p>  
</details>
```

Esta etiqueta no está soportada por todos los navegadores.

1. HTML.

HTML5 ha añadido dos nuevos atributos a esta etiqueta: reversed (booleano) y start.

Attribute	Value	Description
<u>compact</u>	compact	Not supported in HTML5. Specifies that the list should render smaller than normal
<u>reversed</u>	 reversed	Specifies that the list order should be descending (9,8,7...)
<u>start</u>	number	Specifies the start value of an ordered list
<u>type</u>	1 A a I i	Specifies the kind of marker to use in the list

1. HTML.

<dialog>

<!-- <dialog> es una nueva etiqueta html que permite mostrar un diálogo personalizado.

Inicialmente está oculto a menos que le pongamos el atributo open por lo que necesitamos Javascript -->

```
<dialog>
  <button autofocus>Cerrar</button>
  <p>Prueba de etiqueta dialog</p>
</dialog>

<button id="btnM">Mostrar el dialog</button>

<script>
  const dialog = document.querySelector("dialog");
  const open = document.querySelector("dialog + button");
  const close = document.querySelector("dialog button");
  open.addEventListener("click", ()=>{
    dialog.showModal();
  })
  close.addEventListener("click", () =>{
    dialog.close();
  })
</script>
```

1. HTML.

atributo: download

Este atributo para la etiqueta <a> sirve para indicar que el recurso al que apunta debe ser descargado.

Se puede indicar opcionalmente un valor que será el nombre del recurso descargado.

```
<a href="/images/foto.jpg" download">Descargar imagen</a>
```

1. HTML.

<datalist>

<!-- Etiqueta **datalist** asociada a un input con propiedad **list** que hace referencia al id del **datalist** y me permite introducir texto para buscar la opción: -->

```
<input list="lengProg" type="text">
<datalist id="lengProg">
  <option value="C++"></option>
  <option value="Java"></option>
  <option value="PHP"></option>
  <option value="C#"></option>
  <option value="Javascript"></option>
</datalist>
```

```
<h2>HORA</h2>
<input type="time" list="horasTipicas">
<datalist id="horasTipicas">
  <option value="10:00"></option>
  <option value="12:00"></option>
  <option value="14:00"></option>
  <option value="16:00"></option>
</datalist>
```

1. HTML.

atributos: accept y capture en input

`<!-- atributos accept para indicar tipo de fichero y capture para solo en dispositivos móviles seleccionar cámara trasera(con environment) o delantera (Con user) -->`

```
<form action="ejemplo.php" method="post">
  <input type="file" name="image" accept="image/*" capture="user"/>
  <input type="file" name="video" accept="video/*" capture="environment"/>
  <input type="submit" value="upload">
</form>
```

El atributo `capture` en un `<input>` (específicamente de tipo `file`) indica si se debe usar la cámara frontal o trasera de un dispositivo para capturar una imagen o video. Se usa junto con el atributo `accept` y puede tomar los valores `user` (cámara orientada al usuario) o `environment` (cámara orientada al entorno).

- `user`: Solicita que se use la cámara orientada hacia el usuario (frontal).
- `environment`: Solicita que se use la cámara orientada hacia el entorno (trasera).
- Sin valor: Si no se especifica nada, el navegador elegirá la opción predeterminada.

1. HTML.

atributo **popover**

<!-- La propiedad(atributo) **popover** posibilita mostrar un elemento flotante

A un elemento le asignamos la propiedad **popovertarget=id** del elemento con la propiedad popover y su contenido será el popover.

Dando click en cualquier otro lado fuera del popover se cierra-->

```
<style>
  #div1{
    width: 200px;
    height: 100px;
    background: cyan;
  }
</style>
</head>
<body>
  <button popovertarget="div1">Abrir el popover div1</button>
  <div popover id="div1">Saludos desde Benavente</div>
</body>
```

1. HTML.

etiqueta **<search>**

`<body>`

`<!-- La etiqueta search es simplemente semántica y permite definir los
elementos que formarán parte de una búsqueda de cara al SEO para que sepa
que todo el contenido dentro del search hace referencia a búsqueda-->`

`<h2>Ejemplo etiqueta search</h2>`

`<search>`

`<form action="eje.php">`

`<label for="marca">Buscar marca automovil:</label>`

`<input type="search" name="" id="marca">`

`<button type="submit">Buscar marca coche</button>`

`</form>`

`</search>`

`</body>`

2. CSS



CSS



2. CSS

¿QUE ES?

- Cascading Style Sheet
- CSS es un lenguaje de hojas de estilos creado para controlar el aspecto o presentación de los documentos electrónicos definidos con HTML y XHTML.
- CSS es la mejor forma de separar los contenidos y su presentación y es imprescindible para crear páginas web complejas.

2. CSS

ATRIBUTO: HISTORIA

- Con la llegada del HTML 4 se hizo "limpieza" de los elementos que se podían usar, pero, sobre todo, llegó acompañado de la hojas de estilo en cascada.
- Dos propuestas: La de Håkon Wium Lie y la de Bert Bos -> Se unieron por la W3C en -> CSS
- **1996** CSS 1
- **1998** CSS 2
- **2011** CSS 3.1 pero no ha acabado de hacer las recomendaciones
<https://www.w3.org/TR/2018/REC-css-fonts-3-20180920/>

2. CSS

FUNCIÓN

Las hojas de estilo CSS permiten:

- Separar la estructura de las páginas y su contenido (HTML), del formato del texto y de la página
- Tener muchas más posibilidades de formato y de presentación de la página
- Definir un estilo una sola vez y poder aplicarlo tantas veces como se desee
- Evitar los errores causados por la repetición
- Reagrupar todos los estilos
- Hacer actualizaciones de forma extremadamente rápida.

Generador en la web de CSS: <https://css3generator.com/>

2. CSS

FUNCIONAMIENTO

Antes de que se generalizara el uso de CSS, los diseñadores de páginas web utilizaban etiquetas HTML especiales para modificar el aspecto de los elementos de la página

```
<h1><font color="red" face="Arial" size="5">Titular de la página</font></h1>  
<p><font color="gray" face="Verdana" size="2">Un párrafo de texto no muy largo.</font></p>
```

```
h1 {  
  color: red;  
  font-family: Arial;  
  font-size: large;  
}  
p {  
  color: gray;  
  font-family: Verdana;  
  font-size: medium;  
}
```

2. CSS

FUNCIONAMIENTO

- Utilizando CSS, se pueden establecer los mismos estilos con menos esfuerzo y sin ensuciar el código HTML
 - `<style>` en HTML
- Definir los estilos de esta forma ahorra miles de etiquetas y millones de atributos
- Sólo se aplican en la página que los incluye, si queremos que las 10.000 páginas diferentes del sitio tengan el mismo aspecto, se deberían copiar 10.000 veces esas mismas reglas CSS.

2. CSS

INCLUIR CSS EN HTML

En el mismo documento HTML

- Los estilos se definen en una zona específica de head del propio documento HTML .

```
<style>
  p { color: black; font-family: Verdana; }
</style>
```

2. CSS

INCLUIR CSS EN HTML

En un archivo externo

- Los estilos se definen en una zona específica de head del propio documento HTML con la etiqueta link

```
<link rel="stylesheet" href="/css/estilos.css"/>
```

- con la etiqueta style

```
<style type="text/css" media="screen">  
    @import '/css/estilos.css';  
</style>
```

2. CSS

INCLUIR CSS EN HTML

En un los elementos HTML

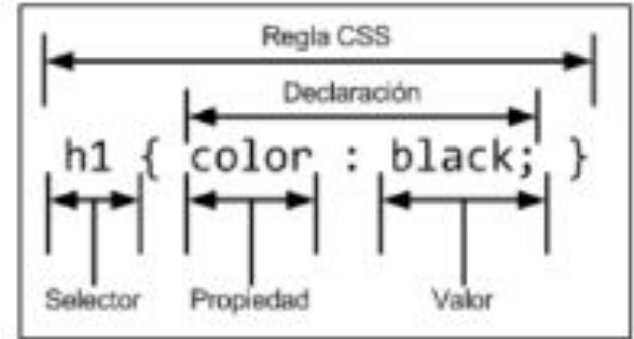
- Los estilos se definen en cada elemento

```
<body>  
  <p style="color: black; font-family: Verdana;">Un párrafo de texto.</p>  
</body>
```

2. CSS

DESCRIPCIÓN DE CSS

- Selector: indica el elemento o elementos HTML a los que se aplica la regla CSS.
- Declaración: especifica los estilos que se aplican a los elementos. Está compuesta por una o más propiedades CSS.
- Propiedad: característica que se modifica en el elemento seleccionado, como por ejemplo su tamaño de letra, su color de fondo, etc.
- Valor: establece el nuevo valor de la característica modificada en el elemento.



2. CSS

MEDIOS CON CSS

CSS permite definir diferentes estilos para diferentes medios o dispositivos: pantallas, impresoras, móviles, proyectores, etc.

Medio	Descripción
all	Todos los medios definidos
braille	Dispositivos táctiles que emplean el sistema braille
embosed	Impresoras braille
handheld	Dispositivos de mano: móviles, PDA, etc.
print	Impresoras y navegadores en el modo "Vista Previa para Imprimir"
projection	Proyectores y dispositivos para presentaciones
screen	Pantallas de ordenador
speech	Sintetizadores para navegadores de voz utilizados por personas discapacitadas
tty	Dispositivos textuales limitados como teletipos y terminales de texto
tv	Televisores y dispositivos con resolución baja

2. CSS

MEDIOS CON CSS

- Reglas de tipo @media

```
@media print {  
    body{font-size: 10pt}  
}  
  
@media screen {  
    body{font-size: 13px}  
}  
  
@media screen, print {  
    body{line-height: 1.2}  
}
```

2. CSS

MEDIOS CON CSS

- Reglas de tipo link

```
<link rel="stylesheet" media="screen" href="basico.css" />  
<link rel="stylesheet" media="print,handheld" href="especial.css" />
```

- Reglas de tipo @import

```
@import url("estilos_basicos.css") screen;  
@import url("estilos_impresora.css") print;
```

2. CSS

COMENTARIOS

CSS permite incluir comentarios entre sus reglas y estilos. Los comentarios son contenidos de texto que el diseñador incluye en el archivo CSS para su propia información y utilidad. Los navegadores ignoran por completo cualquier comentario

```
/* Este es un comentario CSS de  
varias líneas */
```

2. CSS

SELECTORES

En CSS 3 se han introducido nuevos selectores mediante las especificaciones *Selectors Level 3* y *Selectors Level 4*.

En este apartado vemos los selectores ya existentes en CSS 2 y también veremos los nuevos selectores.

https://developer.mozilla.org/es/docs/Web/CSS/Selectores_CSS

2. CSS - Selectores básicos

Selector de tipo

Selecciona todos los elementos que coinciden con el nombre del elemento especificado.

Sintaxis: `nombreElemento`

Ejemplo: `input` se aplicará a cualquier elemento `<input>`.

Selector de clase

Selecciona todos los elementos que tienen el atributo de `class` especificado.

Sintaxis: `.classnamebre`

Ejemplo: `.index` seleccionará cualquier elemento que tenga la clase `"index"`.

Selector de ID

Selecciona un elemento por el valor de su atributo `id`. Solo puede haber un elemento con un determinado ID dentro de un documento.

Sintaxis: `#idnombre`

Ejemplo: `#toc` se aplicará a cualquier elemento que tenga el ID `"toc"`.

Selector universal

Selecciona todos los elementos. Opcionalmente, puede estar restringido a un espacio de nombre específico o a todos los espacios de nombres.

Sintaxis: `* ns | * * | *`

Ejemplo: `*` se aplicará a todos los elementos del documento.

Selector de atributo

Selecciona elementos basándose en el valor de un determinado atributo.

Sintaxis: `[attr]` `[attr=value]` `[attr~=value]`
`[attr|=value]` `[attr^=value]` `[attr$=value]`
`[attr*=value]`

Ejemplo: `[autoplay]` seleccionará todos los elementos que tengan el atributo `"autoplay"` establecido (a cualquier valor).

2. CSS - Combinadores

Combinador de hermanos adyacentes +

El combinador + selecciona hermanos adyacentes. Esto quiere decir que el segundo elemento sigue directamente al primero y ambos comparten el mismo elemento padre.

Sintaxis: A + B

Ejemplo: La regla h2 + p se aplicará a todos los elementos <p> que siguen directamente a un elemento <h2>.

Combinador general de hermanos ~

El combinador ~ selecciona hermanos. Esto quiere decir que el segundo elemento sigue al primero (no necesariamente de forma inmediata) y ambos comparten el mismo elemento padre.

Sintaxis: A ~ B

Ejemplo: La regla p ~ span se aplicará a todos los elementos que siguen un elemento <p>.

Combinador de hijo >

El combinador > selecciona los elementos que son hijos directos del primer elemento.

Sintaxis: A > B

Ejemplo: La regla ul > li se aplicará a todos los elementos que son hijos directos de un elemento .

Combinador de descendientes

El combinador (**espacio**) selecciona los elementos que son descendientes del primer elemento.

Sintaxis: A B

Ejemplo: La regla div span se aplicará a todos los elementos que están dentro de un elemento <div>.

2. CSS - Pseudoclasses

Las **pseudoclasses** permiten la selección de elementos, basada en información de estado que no está contenida en el árbol de documentos.

Ejemplo: La regla `a:visited` se aplicará a todos los elementos `<a>` que hayan sido visitados por el usuario.

Una **pseudoclase CSS** es una palabra clave que se añade a los selectores y que especifica un estado especial del elemento seleccionado. Por ejemplo, `:hover` aplicará un estilo cuando el usuario haga hover sobre el elemento especificado por el selector.

```
selector:pseudoclase { propiedad: valor; }
```

2. CSS - Pseudoclasses

Índice de las pseudo-clases estándar

- `active`
- `:checked`
- `:default`
- `:dir()`
- `:disabled`
- `:empty`
- `:enabled`
- `:first`
- `:first-child`
- `:first-of-type`
- `:fullscreen`
- `:focus`
- `:hover`
- `:has()`
- `:indeterminate`
- `:in-range`
- `:invalid`
- `:lang()`
- `:last-child`
- `:last-of-type`
- `:left`
- `:link`
- `:not()`
- `:nth-child()`
- `:nth-last-child()`
- `:nth-last-of-type()`
- `:nth-of-type()`
- `:only-child`
- `:only-of-type`
- `:optional`
- `:out-of-range`
- `:read-only`
- `:read-write`
- `:required`
- `:right`
- `:root`
- `:scope`
- `:target`
- `:valid`
- `:visited`

2. CSS - Pseudoelementos

Pseudoelementos

Los **pseudoelementos** son abstracciones del árbol que representan entidades más allá de los elementos HTML. Por ejemplo, HTML no tiene un elemento que describa la primera letra de un párrafo ni los marcadores de una lista. Los pseudoelementos representan estas entidades y nos permiten asignarles reglas CSS.

Ejemplo: La regla **p::first-line** se aplicará a la primera línea de texto de todos los elementos `<p>`.

Lista de pseudoelementos

- `::after`
- `::before`
- `::first-letter`
- `::first-line`
- `::selection`
- `::backdrop`
- `::placeholder`
- `::marker`
- `::spelling-error`
- `::grammar-error`

2. CSS

SELECTORES

Selectores básicos -> Selectores universal *

Se utiliza para seleccionar todos los elementos de la página. El siguiente ejemplo elimina el margen y el relleno de todos los elementos HTML

```
* {  
    margin: 0;  
    padding: 0;  
}
```

2. CSS

SELECTORES

Selectores básicos -> Selectores de tipo o etiqueta

Selecciona todos los elementos de la página cuya etiqueta HTML coincide con el valor del selector:

```
h1 {  
    color: red;  
}  
  
h2 {  
    color: blue;  
}
```

2. CSS

SELECTORES

Selectores básicos -> Selectores de tipo o etiqueta

En este caso, CSS permite agrupar todas las reglas individuales en una sola regla con un selector múltiple. Para ello, se incluyen todos los selectores separados por una coma (,):

```
h1, h2, h3 {  
    color: #8A8E27;  
    font-weight: normal;  
    font-family: Arial, Helvetica, sans-serif;  
}
```

2. CSS

SELECTORES

Selectores básicos -> Selectores de tipo o etiqueta

En las hojas de estilo complejas, es habitual agrupar las propiedades comunes de varios elementos en una única regla CSS y posteriormente definir las propiedades específicas de esos mismos elementos.

```
h1, h2, h3 {  
  color: #8A8E27;  
  font-weight: normal;  
  font-family: Arial, Helvetica, sans-serif;  
}
```

```
h1 { font-size: 2em; }  
h2 { font-size: 1.5em; }
```

2. CSS

SELECTORES

Selectores básicos -> Selectores descendente

Selecciona los elementos que se encuentran dentro de otros elementos. Un elemento es descendiente de otro cuando se encuentra entre las etiquetas de apertura y de cierre del otro elemento.

```
p span { color: red; }
```

```
<p>  
  <span>texto1</span>  
  <a href="">...<span>texto2</span></a>  
</p>
```

2. CSS

SELECTORES

Selectores básicos -> Selectores descendente

Se puede restringir el alcance del selector descendente combinándolo con el selector universal.

```
p a { color: red; }  
<p><a href="#">Enlace</a></p>  
<p><span><a href="#">Enlace</a></span></p>
```

Enlace

Enlace

```
p * a { color: red; }  
<p><a href="#">Enlace</a></p>  
<p><span><a href="#">Enlace</a></span></p>
```

Enlace

Enlace

2. CSS

SELECTORES

Selectores básicos -> Selectores de clase

```
<body>  
  <p class="destacado">Lorem ipsum dolor sit...</p>  
  <p>Nunc sed lacus et est adipiscing accumsan...</p>  
  <p>Class aptent taciti sociosqu ad litora...</p>  
</body>
```

Para que el navegador no confunda este selector con los otros tipos de selectores, se prefija el valor del atributo `class` con un punto (.)

```
.destacado { color: red; }
```

2. CSS

SELECTORES

Selectores básicos -> Selectores de clase

```
<body>  
  <p class="destacado">Lorem ipsum dolor sit...</p>  
  <p>Nunc sed lacus et est adipiscing accumsan...</p>  
  <p>Class aptent taciti <em class="destacado">sociosqu  
ad</em> ad litora...</p>  
</body>
```

```
p.destacado { color: red; }
```

2. CSS

SELECTORES

Selectores básicos -> Selectores de clase

Ejemplos

```
.aviso {  
  padding: 0.5em;  
  border: 1px solid #98be10;  
  background: #f6feda;  
}  
  
.error {  
  color: #930;  
  font-weight: bold;  
}  
<span class="error">...</span>  
  
<div class="aviso">...</div>
```

2. CSS

SELECTORES

Selectores básicos -> Selectores de clase

```
/* Todos los elementos de tipo "p" con atributo class="aviso" */  
p.aviso { ... }
```

```
/* Todos los elementos con atributo class="aviso" que estén dentro  
de cualquier elemento de tipo "p" */  
p .aviso { ... }
```

```
/* Todos los elementos "p" de la página y todos los elementos con  
atributo class="aviso" de la página */  
p, .aviso { ... }
```

2. CSS

SELECTORES

Selectores básicos -> Selectores de clase

Varias clases en un elemento

```
.error { color: red; }  
.destacado { font-size: 15px; }  
.especial { font-weight: bold; }
```

```
<p class="especial destacado error">Párrafo de texto...</p>
```

El color de la letra del texto es azul y no rojo.

```
.error.destacado { color: blue; }
```

2. CSS

SELECTORES

Selectores básicos -> Selectores de id

El selector de ID permite seleccionar un elemento de la página a través del valor de su atributo id. Este tipo de selectores sólo seleccionan un elemento de la página porque el valor del atributo id no se puede repetir en dos elementos diferentes de una misma página.

La sintaxis de los selectores de ID es muy parecida a la de los selectores de clase, salvo que se utiliza el símbolo de la almohadilla (#) en vez del punto (.)

```
#destacado { color: red; }
```

```
<p>Primer párrafo</p>
```

```
<p id="destacado">Segundo párrafo</p>
```

```
<p>Tercer párrafo</p>
```

2. CSS

ATRIBUTO: SELECTORES

Selectores básicos -> Selectores de id

Al igual que los selectores de clase, en este caso también se puede restringir el alcance del selector mediante la combinación con otros selectores.

```
p#aviso { color: blue; }
```

```
/* Todos los elementos de tipo "p" con atributo id="aviso" */  
p#aviso { ... }
```

```
/* Todos los elementos con atributo id="aviso" que estén dentro  
   de cualquier elemento de tipo "p" */  
p #aviso { ... }
```

```
/* Todos los elementos "p" de la página y todos los elementos con  
   atributo id="aviso" de la página */  
p, #aviso { ... }
```

2. CSS

SELECTORES

Selectores básicos -> Combinación de selectores básicos

CSS permite la combinación de uno o más tipos de selectores para restringir el alcance de las reglas CSS. A continuación se muestran algunos ejemplos habituales de combinación de selectores.

```
.aviso .especial { ... }
```

Si se modifica el anterior selector:

```
div.aviso span.especial { ... }
```

2. CSS

SELECTORES

Selectores básicos -> Combinación de selectores básicos

La combinación de selectores puede llegar a ser todo lo compleja que sea necesario:

```
ul#menuPrincipal li.destacado a#inicio { ... }
```

2. CSS

ATRIBUTO: SELECTORES

Selectores avanzados (Combinadores)

Utilizando solamente los selectores básicos de la sección anterior, es posible diseñar prácticamente cualquier página web. No obstante, CSS define otros selectores más avanzados que permiten simplificar las hojas de estilos.

- [Lista completa de los selectores que soporta cada versión de cada navegador](#)
- [Test online en el que puedes comprobar los selectores que soporta el navegador con el que realizas el test](#)

2. CSS

SELECTORES

Selector de hijos

El siguiente ejemplo muestra las diferencias entre el selector descendente y el selector de hijos:

```
p a { color: red; }  
p > a { font-style: italic; }  
p * a { background: grey; }
```

```
<p><a href="#">Enlace1</a></p>
```

Enlace1

```
<p><span><a href="#">Enlace2</a></span></p>
```

Enlace2

```
<p><a href="#">Enlace1</a></p>
```

Enlace1

2. CSS

SELECTORES

Selector de hijos >

Se trata de un selector similar al selector descendente, pero muy diferente en su funcionamiento. Se utiliza para seleccionar un elemento que es *hijo directo* de otro elemento y se indica mediante el "*signo de mayor que*" (>):

```
p > span { color: red; }
```

```
<p><span>Texto1</span></p>
```

Texto1

```
<p><a href="#"><span>Texto2</span></a></p>
```

Texto2

```
<p><span>Texto3</span></p>
```

Texto3

2. CSS

SELECTORES

Selector hermanos adyacentes +

El selector adyacente se emplea para seleccionar elementos que en el código HTML de la página se encuentran justo a continuación de otros elementos. Su sintaxis emplea el signo + para separar los dos elementos:

```
elemento1 + elemento2 { ... }
```

```
h2 { color: green; }  
h1 + h2 { color: red; }
```

```
<h1>Titulo1</h1>  
<h2>Subtítulo</h2>  
<p> Lorem </p>  
<h2>Otro subtítulo</h2>
```

Titulo1

Subtítulo

Lorem

Otro subtítulo

2. CSS

ATRIBUTO: SELECTORES

Selector hermanos adyacentes +

Técnicamente, los elementos que forman el selector adyacente deben cumplir las dos siguientes condiciones:

- elemento1 y elemento2 deben ser elementos hermanos, por lo que su elemento padre debe ser el mismo.
- elemento2 debe aparecer inmediatamente después de elemento1 en el código HTML de la página.

```
p + p { text-indent: 1.5em; }
```

2. CSS

SELECTORES

Selector hermanos ~

Selecciona todos los elementos de la página de tipo h2 que son hermanos de un elemento de tipo a el cual les precede en el código.

```
h1 ~ h2 { color: red }
```

```
<header>
  <h1>Titulo1</h1>
  <h2>Subtítulo</h2>
</header>
<p> Lorem </p>
<h2>Otro subtítulo</h2>
```

Titulo1

Subtítulo

Lorem

Otro subtítulo

2. CSS

SELECTORES

Selector de atributos

permiten seleccionar elementos HTML en función de sus atributos y/o valores de esos atributos.

sintaxis	Descripción
[atribut]	Selecciona el elemento con este atributo.
[atribut="valor"]	El valor del atributo es valor.
[atribut^="text"]	El valor del atributo comienza por texto.
[atribut\$="text"]	El valor del atributo acaba con texto.
[atribut*="text"]	El valor del atributo contiene el texto.
[atribut~="valor"]	El valor del atributo es una lista de palabras separadas por espacios, una de las cuales es exactamente valor.
[atribut ="valor"]	El valor del atributo es una lista de palabras separadas por guiones, empezando por valor.

2. CSS

SELECTORES

Selector de atributos

[nombre_atributo] selecciona los elementos que tienen establecido el atributo llamado nombre_atributo, independientemente de su valor.

```
h2[class] { color: blue; }
```

```
/* Se muestran de color azul todos los enlaces que tengan  
un atributo "class", independientemente de su valor */
```

2. CSS

SELECTORES

Selector de atributos

[nombre_atributo=valor], selecciona los elementos que tienen establecido un atributo llamado nombre_atributo con un valor igual a valor.

```
/* Se muestran de color azul todos los enlaces que  
tengan  
un atributo "class" con el valor "externo" */
```

```
a[class="externo"] { color: blue; }
```

2. CSS

SELECTORES

Selector de atributos

[nombre_atributo~=valor], selecciona los elementos que tienen establecido un atributo llamado nombre_atributo y al menos uno de los valores del atributo es valor.

```
/* Se muestran de color azul todos los enlaces que tengan  
un atributo "class" en el que al menos uno de sus  
valores  
sea "externo" */
```

```
a[class~="externo"] { color: blue; }
```

2. CSS

SELECTORES

Selector de atributos

[nombre_atributo|=valor], selecciona los elementos que tienen establecido un atributo llamado nombre_atributo y cuyo valor es una serie de palabras separadas con guiones, pero que comienza con valor. Este tipo de selector sólo es útil para los atributos de tipo lang que indican el idioma del contenido del elemento.

```
/* Selecciona todos los elementos de la página cuyo atributo  
   "lang" empiece por "es", es decir, "es", "es-ES", "es-AR", etc. */
```

```
*[lang|= "es"] { color : red }
```

2. CSS

SELECTORES

Selector de atributos

[nombre_atributo^=valor], selecciona los elementos que tienen establecido un atributo llamado nombre_atributo y empieza por valor.

```
/* Selecciona todos los elementos de la página cuyo  
atributo empiece por "caja*/  
*[class^="caja"] { color : red }
```

2. CSS

ATRIBUTO: SELECTORES

Selector de atributos

[nombre_atributo\$=valor], selecciona los elementos que tienen establecido un atributo llamado nombre_atributo y cuyo valor termina por valor.

```
/* Selecciona todos los elementos de la página cuyo  
atributo acabe por "ces" */  
*[class$="ces"] { color : red }
```

2. CSS

HERENCIA

Una de las características principales de CSS es la herencia de los estilos definidos para los elementos. Cuando se establece el valor de una propiedad CSS en un elemento, sus elementos descendientes heredan de forma automática el valor de esa propiedad.

```
<style type="text/css">
  body { color: blue; }
</style>
</head>

<body>
  <h1>Titular de la página</h1>
  <p>Un párrafo de texto no muy largo.</p>
</body>
</html>
```

Titular de la página

Un párrafo de texto no muy largo.

2. CSS

HERENCIA

Aunque la herencia de estilos se aplica automáticamente, se puede anular su efecto estableciendo de forma explícita otro valor para la propiedad que se hereda

```
<style type="text/css">
  body { font-family: Arial; color: black; }
  h1 { font-family: Verdana; }
  p { color: red; }
</style>
```

```
<body>
  <h1>Titular de la página</h1>
  <p>Un párrafo de texto no muy largo.</p>
</body>
```

Titular de la página

Un párrafo de texto no muy largo.

2. CSS

COLISIONES

En las hojas de estilos complejas, es habitual que varias reglas CSS se apliquen a un mismo elemento HTML. El problema de estas reglas múltiples es que se pueden dar colisiones como la del siguiente ejemplo:

```
p { color: red; }  
p { color: blue; }
```

1. Cuanto más específico sea un selector, más importancia tiene su regla asociada. Podemos poner `!important`
2. A igual *especificidad*, se considera la última regla indicada.

2. CSS

ESPECIFICIDAD CSS

1. Es el método por el cual el navegador determina que estilos tienen mayor peso o relevancia de aplicación por sobre otros. El orden es así:

Elemento	Nivel
etiquetas	0
clases, de atributos y pseudoclases	1
selectores de <code>id</code>	2

Vemos que es más específico usar la pseudoclase `:root` que la etiqueta `<html>`.

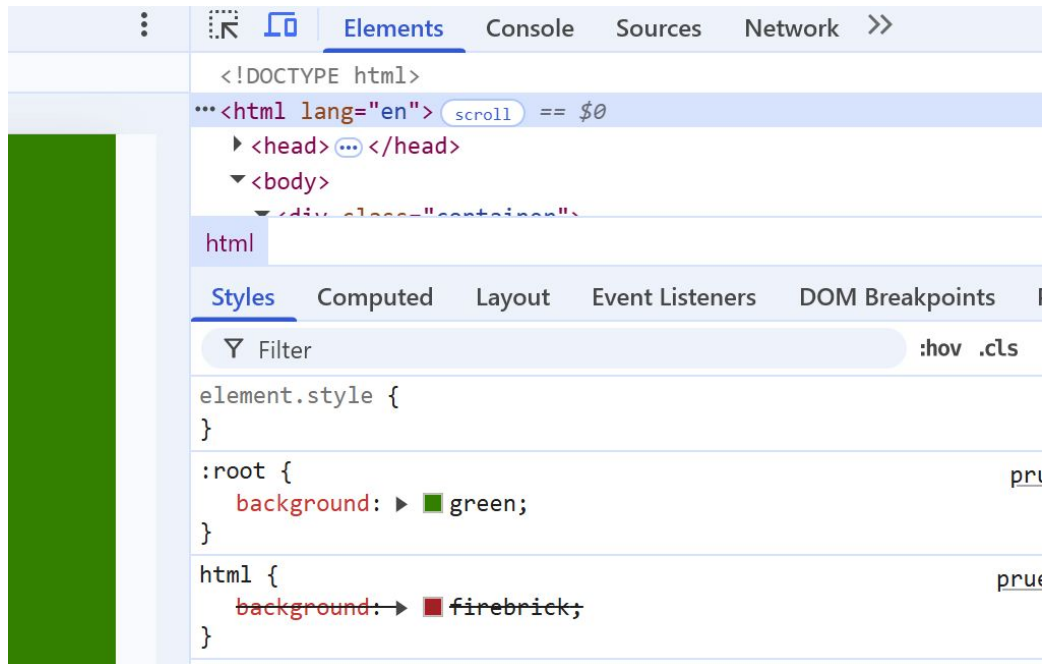
Se puede comprobar en la siguiente imagen del navegador:

2. CSS

ESPECIFICIDAD CSS

Elemento	Nivel
etiquetas	0
clases, de atributos y pseudoclases	1
selectores de id	2

```
<style>
:root{
  background: green;
}
html{
  background: firebrick;
}
```



The screenshot shows a web browser's developer tools interface. The 'Elements' panel displays the DOM tree, starting with the root element 'html' (lang="en"). The 'Styles' panel shows the 'element.style' rule with 'background: green;' and the 'html' rule with 'background: firebrick;'. The 'html' rule is highlighted, indicating it is the active style. The 'background' property is set to 'green' for the 'element.style' rule and 'firebrick' for the 'html' rule.

2. CSS

COLISIONES

```
<style>
  p { color: red; }
  p#especial { color: green; }
  * { color: blue; }
</style>
```

```
<p id="especial">Que sera, sera...</p>
```

Que sera, sera...

Ej:

```
div{
  background: url(fondoNuevo.png) !important;
  background: url(fondo.gif);
}
```

2. CSS

PSEUDO-SELECTORES: PSEUDO-CLASE

ELEMENTOS DE TIPO <A>

Clasifican a los elementos basándose en características que van más allá de su nombre, atributos o contenido. Define un estilo diferente en función del comportamiento del enlace: si todavía no ha sido visitado (link), si ya lo ha sido (visited), mientras el ratón pasa por encima (hover) o justo cuando se pulsa el ratón sobre él (active).

```
a:link {color: red;}  
a:visited {color: blue;}  
a:hover {color: fuchsia;}  
a:active {color: maroon;}
```

2. CSS

PSEUDO-SELECTORES: PSEUDO-CLASE

pseudoclase	Descripción
:root	Representa un elemento que se encuentra en la raíz del documento.
:nth-child (num)	Selecciona el elemento indicado pero con la condición de que sea el hijo enésimo de su padre.
:nth-last-child (num)	Idéntico al anterior, pero el número indicado se empieza a contar desde el último hijo.
:nth-of-type (num)	Selecciona el elemento indicado pero con la condición de que sea el enésimo elemento hijo de este tipo.
:nth-last-of-type (num)	Idéntico al anterior, pero el número indicado se empieza a contar desde el último hijo.
:first-child	Selecciona un elemento que sea el primer hijo de otro elemento.
:last-child	Selecciona el elemento indicado, pero con la condición de que sean el último hijo de su elemento padre.
:first-of-type	Hace referencia al primer elemento de este tipo en el elemento padre.
:last-of-type	Hace referencia al último elemento de este tipo en el elemento padre.
:only-of-type	Hace referencia a aquel elemento que es el único hijo de su padre.
:empty	Selecciona el elemento indicado pero con la condición de que no tenga ningún hijo y tampoco puede tener ningún contenido de texto.
:lang	De idioma. Permite aplicar estilos en función del idioma especificado.
:not	De negación. Seleccionar todos los elementos que no cumplen con la condición de un selector.

2. CSS

PSEUDO-SELECTORES: PSEUDO-CLASE

`*:empty {}`

`/* Selecciona cualquier <div> que esté vacío*`

```
div: empty{ background : blue;  
            width: 100px;  
            height: 20px;}
```

Este div `<div></div>` está vacío

2. CSS

PSEUDO-SELECTORES: PSEUDO-CLASE

:root{}

/* selecciona el elemento raíz de un árbol que representa el documento.

```
:root { background-color: #10eb2e; }
```

Diferencia entre **:root** y *****

```
* { background-color: #10eb2e; }
```

Los estilos que se aplican en **:root** solo afectan al elemento `<html>` y los del selector asterisco ***** afectan a todos los elementos, lo que lo hace mucho más pesado para el navegador.

2. CSS

PSEUDO-SELECTORES: PSEUDO-CLASE

***:first-child {}**

/* Selecciona el primer hijo de section que sea p*

```
p: first-child{  
    color: blue;  
}
```

```
<section>  
    <p>First Child</p>  
    <h1>Child</h1>  
    <p>Child</p>  
    <p>Child</p>  
</section>
```

2. CSS

PSEUDO-SELECTORES: PSEUDO-CLASE

***:first-of-type {}**

```
/* Selecciona el primer hijo de tipo p*  
p:first-of-type{  
    color: blue;  
}  
<section>  
    <h1>Child</h1>  
    <p>Child</p> ←  
    <p>Child</p>  
    <p>Child</p>  
</section>
```

¿Es igual que **first-child**? ¿Y si quiero seleccionar el primer elemento de cada tipo?

2. CSS

PSEUDO-SELECTORES: PSEUDO-CLASE

***:lang{ }**

/* Se emplea para seleccionar elementos en función de su idioma. El idioma está determinado por los atributos lang*/

```
p { color: blue; }
```

```
p:lang(es) { color: red; }
```

```
<p lang="en">Lorem ipsum dolor sit amet...</p>
```

```
<div lang="fr">
```

```
<p>Lorem ipsum dolor sit amet...</p>
```

```
<p lang="es_ES">Lorem ipsum dolor sit amet...</p>
```

```
</div>
```

2. CSS

PSEUDO-SELECTORES: PSEUDO-CLASE

***:last-child**

/* Se emplea para seleccionar todos los elementos que sean el último hijo de otro elemento*/

```
h1: last-child{  
  color: blue;  
}  
  
<section>  
  <h1>Child</h1>  
  <p>Child</p>  
  <p>Child</p>  
  <p>Child</p>  
</section>
```

2. CSS

PSEUDO-SELECTORES: PSEUDO-CLASE

***:last-of-type**

/* Seleccionar todos los elementos que sean el último hijo de otro elemento*/

```
section : last-of-type{  
  color: blue;  
}  
<section>  
  <h1>Child</h1>  
  <p>Child</p>  
  <p>Child</p>  
  <p>Child</p>  
</section>
```

2. CSS

PSEUDO-SELECTORES: PSEUDO-CLASE

***:nth-child(n)**

/* Selecciona el elemento el hijo enésimo de su padre. Admite parámetros de la forma:

(2n): hijos múltiplos de 2

(3n+4): múltiplos de 3 a partir del cuarto

(odd): impares

(even): pares*/

```
p: nth-child(odd) {  
  color: blue;  
}
```

```
<section>  
  <h1>Child</h1>  
  <p>Child</p>  
  <p>Child</p>  
  <p>Child</p>  
</section>
```

2. CSS

PSEUDO-SELECTORES: PSEUDO-CLASE

***:last-child(n)**

/* Selecciona el elemento indicado pero con la condición de que sea el hijo enésimo de su padre.

Admite parámetros de la forma:

(2n): hijos múltiplos de 2

(3n+4): múltiplos de 3 a partir del cuarto

(odd): impares

(even): pares*/

```
section:last-child(even) {  
  color: blue; }
```

```
<section>  
  <h1>Child</h1>  
  <p>Child</p>  
  <p>Child</p>  
  <p>Child</p>  
</section>
```

2. CSS

PSEUDO-SELECTORES: PSEUDO-CLASE

***:nth-of-type()**

/* Selecciona el elemento indicado pero con la condición de que sea el hijo enésimo de su padre. Admite parámetros de la forma:

(2n): hijos múltiplos de 2

(3n+4): múltiplos de 3 a partir del cuarto

(odd): impares

```
(even): pares*/  
p : nth-of-type(3) { color: blue; }  
<section>  
  <h1>Child</h1>  
  <p>Child</p>  
  <p>Child</p>  
  <p>Child</p>  
</section>
```

2. CSS

*diferencias :nth-child y :nth-of-type

- :nth-child selecciona el elemento que es el secundario de su elemento primario.
- :nth-of-type selecciona el elemento que es el enésimo elemento del mismo nivel de su tipo.

Ejemplo:

- `<section class="datos">`
- `<h1>Información</h1>`
- `<h2>Datos personales</h2>`
- `<p>Nombre</p>`
- `<p>Apellidos</p>`
- `<p>Teléfono</p>`
- `<p>Dirección</p>`
- `</section>`

Y queremos seleccionar el elemento (p) que contiene el texto de «Teléfono» (es el 3º párrafo) y aplicarle un color azul:

- `.datos p:nth-child(3) { color: blue; }/* MAL*/`

Esta forma sería incorrecta. Con `p:nth-child(3)`, lo que hacemos es seleccionar el 3º elemento de su padre (`.datos`) que en este caso es el campo con texto «Nombre».

- `.datos p:nth-of-type(3) { color: blue; }/*BIEN */`

Y en cambio con `p:nth-of-type(3)` seleccionamos el 3º elemento del mismo tipo (párrafo) de su elemento padre (`.datos`)

2. CSS

PSEUDO-SELECTORES: PSEUDO-CLASE

***:only-child**

```
/* Selecciona el elemento que no tenga hermano*/
```

```
a:only-child{  
    color: blue; }
```

```
<section>  
  <h1>Child</h1>  
  <p><a>Child</a></p>  
  <p>Child</p>  
  <p>Child</p>  
</section>
```

2. CSS

PSEUDO-SELECTORES: PSEUDO-CLASE

:target{...}

Es empleado para dar estilo al elemento HTML actualmente activado

```
<ol>
  <li><a href="#p1">¡Salta al primer párrafo!</a></li>
  <li><a href="#p2">¡Salta al segundo párrafo!</a></li>
</ol>
```

```
<p id="p1">Lorem ipsum dolor sit amet consectetur adipiscing elit quisque erat</p>
<p id="p2">Praesent neque a varius eros turpis lectus per vestibulum dictumst</p>
```

```
p:target {
  background-color: grey;
}
```

2. CSS

PSEUDO-SELECTORES: PSEUDO-CLASE

ELEMENTOS DE TIPO <INPUT>

`input:checked`

`input:disabled`

`input:enabled`

`input:focus`

`input:in-range`

`input:invalid`

`input:optional`

`input:out-of-range`

`input:read-only`

`input:read-write`

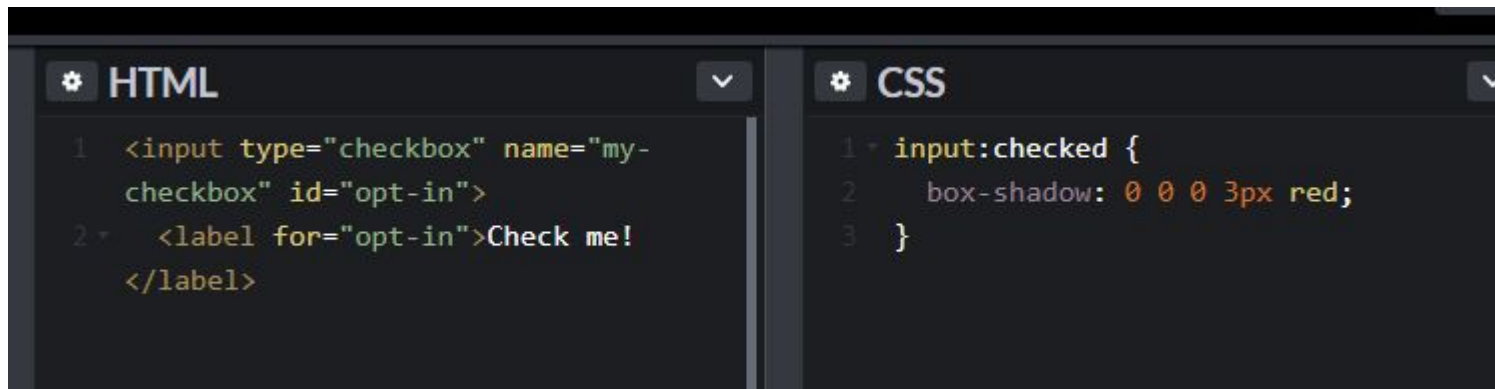
`input:required`

`input:valid`

2. CSS

PSEUDO-SELECTORES: PSEUDO-CLASE

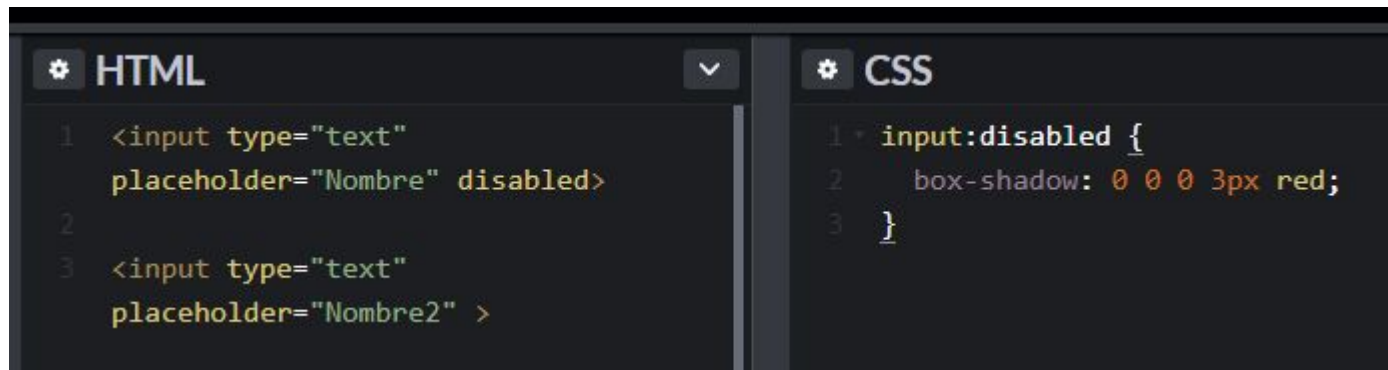
input:checked{..}



2. CSS

PSEUDO-SELECTORES: PSEUDO-CLASE

input:disabled{..}



The image shows a screenshot of a code editor with two panels. The left panel is titled 'HTML' and contains the following code:

```
1 <input type="text"  
  placeholder="Nombre" disabled>  
2  
3 <input type="text"  
  placeholder="Nombre2" >
```

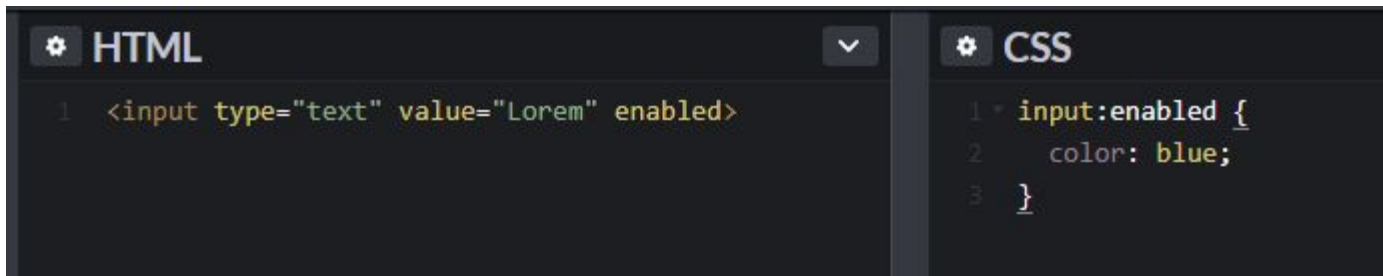
The right panel is titled 'CSS' and contains the following code:

```
1 * input:disabled {  
2   box-shadow: 0 0 0 3px red;  
3 }
```

2. CSS

PSEUDO-SELECTORES: PSEUDO-CLASE

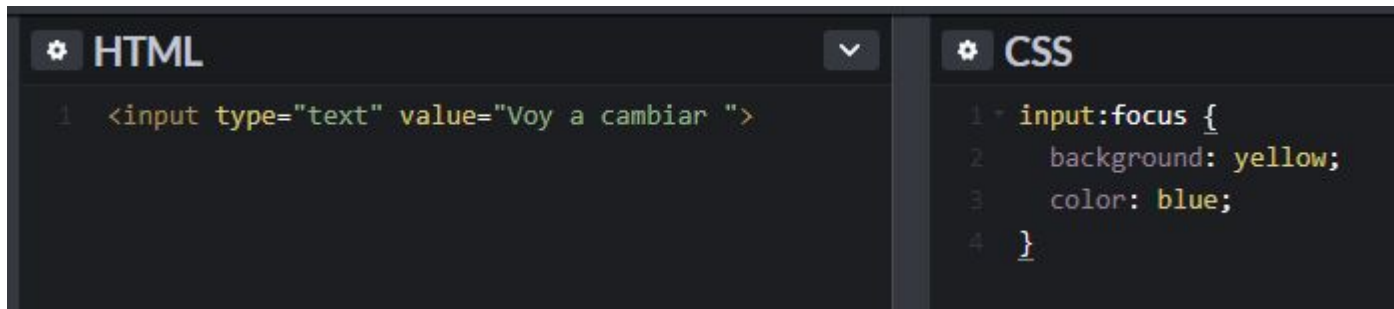
input:enabled{..}



2. CSS

PSEUDO-SELECTORES: PSEUDO-CLASE

input:focus{..}



The image shows a screenshot of a code editor with two panels. The left panel is titled 'HTML' and contains a single line of code: `<input type="text" value="Voy a cambiar ">`. The right panel is titled 'CSS' and contains a CSS rule for the `input:focus` pseudo-class, setting the background to yellow and the color to blue.

```
HTML
1 <input type="text" value="Voy a cambiar ">

CSS
1 input:focus {
2     background: yellow;
3     color: blue;
4 }
```

2. CSS

PSEUDO-SELECTORES: PSEUDO-CLASE

input:in-range{..}



```
HTML
1 <input name="value1" type="number"
  placeholder="1 a 10" min="1" max="10" value="12">
2

CSS
1 input:in-range {
2     background-color: lime;
3 }
```

2. CSS

PSEUDO-SELECTORES: PSEUDO-CLASE

input:out-of-range{..}

HTML

```
1 <input name="value1" type="number"  
  placeholder="1 a 10" min="1" max="10" value="12">  
2
```

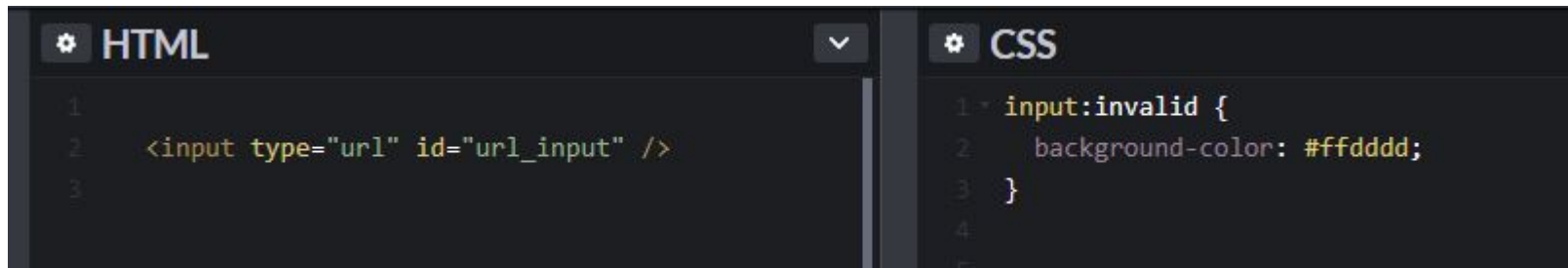
CSS

```
1 * input:in-range {  
2   background-color: lime;  
3 }  
4  
5 * input:out-of-range {  
6   background-color: red;  
7 }
```

2. CSS

PSEUDO-SELECTORES: PSEUDO-CLASE

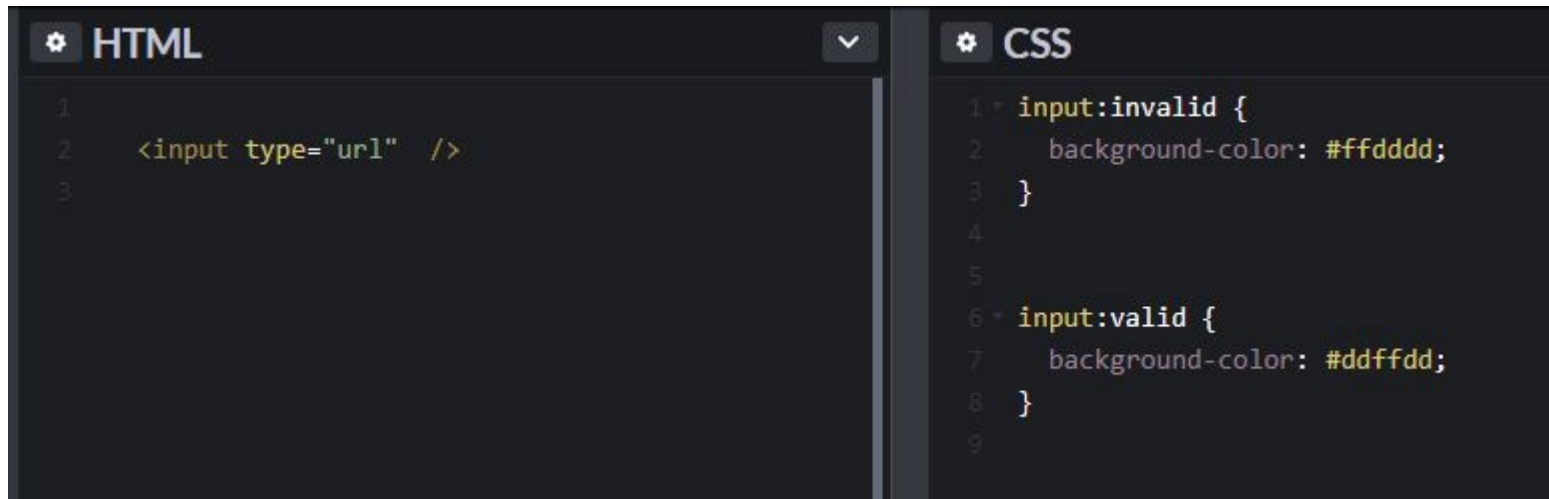
input:invalid{..}



2. CSS

PSEUDO-SELECTORES: PSEUDO-CLASE

input:valid{..}



The image shows a code editor with two panels. The left panel is titled 'HTML' and contains the following code:

```
1
2  <input type="url" />
3
```

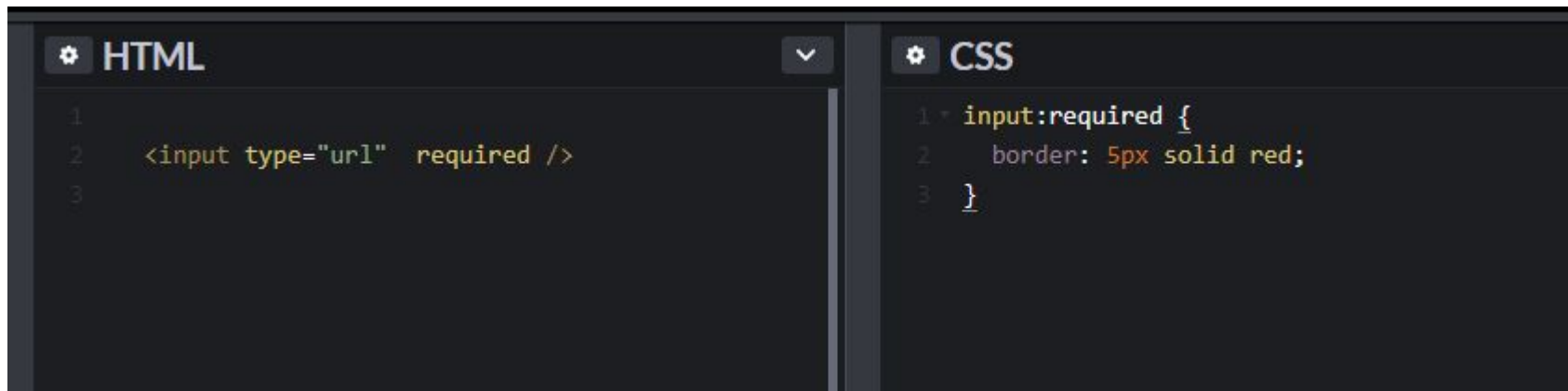
The right panel is titled 'CSS' and contains the following code:

```
1 * input:invalid {
2   background-color: #ffdddd;
3 }
4
5
6 * input:valid {
7   background-color: #ddffdd;
8 }
9
```

2. CSS

PSEUDO-SELECTORES: PSEUDO-CLASE

input:required{..}



The image shows a screenshot of a code editor with two panels. The left panel is titled 'HTML' and contains the following code:

```
1  
2 <input type="url" required />  
3
```

The right panel is titled 'CSS' and contains the following code:

```
1 input:required {  
2   border: 5px solid red;  
3 }
```

2. CSS

PSEUDO-SELECTORES: PSEUDO-CLASE

input:optional{..}

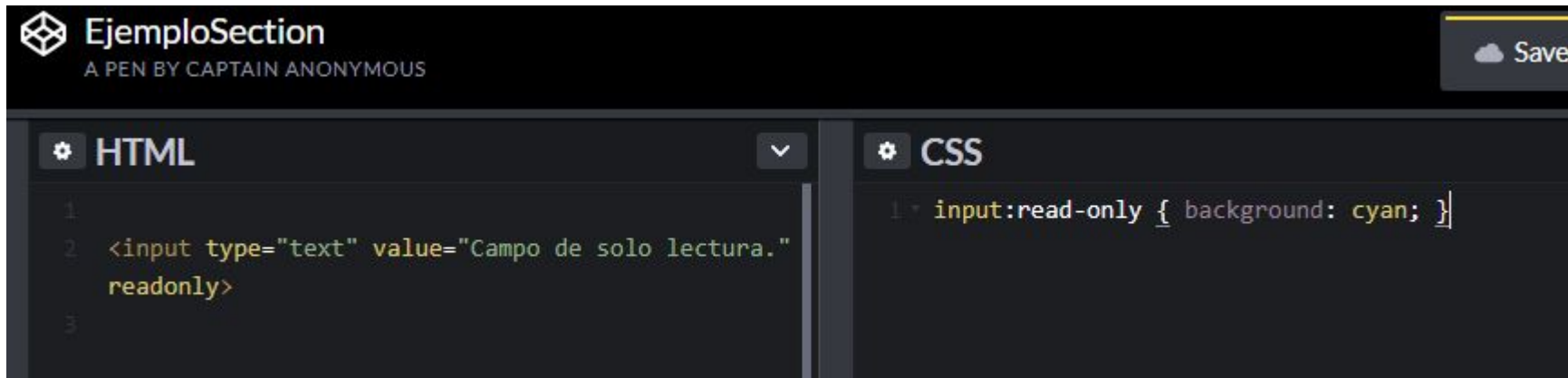
```
<input type="url" required />  
<input type="url" />
```

```
1  
2 input:required {  
3   border: 5px solid red;  
4 }  
5 input:optional {  
6   border: 5px solid lime;  
7 }
```

2. CSS

PSEUDO-SELECTORES: PSEUDO-CLASE

input:read-only{..}



The screenshot shows a code editor interface with a dark theme. At the top left, there is a logo and the text "EjemploSection" and "A PEN BY CAPTAIN ANONYMOUS". At the top right, there is a "Save" button with a cloud icon. The editor is split into two panels. The left panel is titled "HTML" and contains the following code:

```
1  
2 <input type="text" value="Campo de solo lectura."  
   readonly>  
3
```

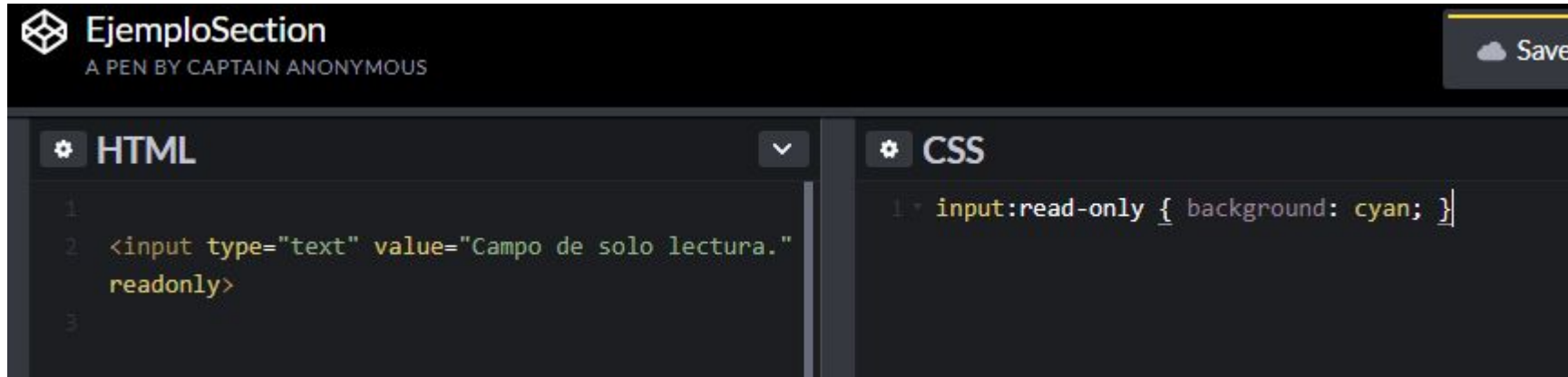
The right panel is titled "CSS" and contains the following code:

```
1 input:read-only { background: cyan; }
```

2. CSS

PSEUDO-SELECTORES: PSEUDO-CLASE

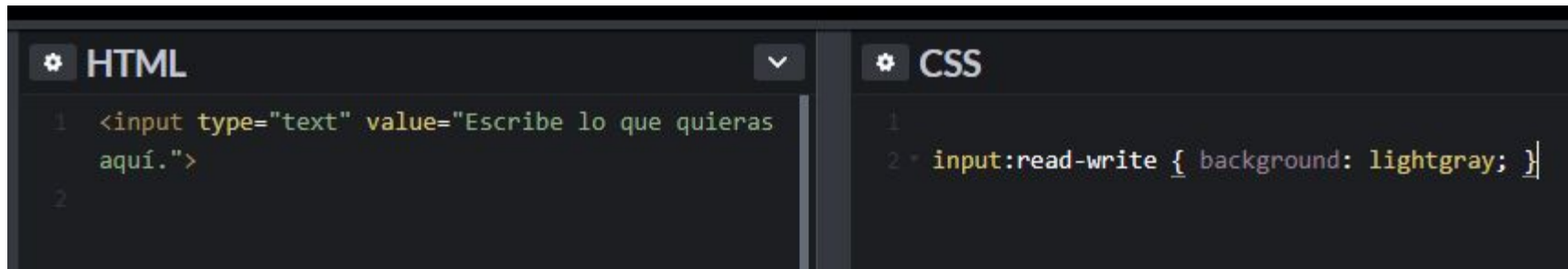
input:read-only{..}



2. CSS

PSEUDO-SELECTORES: PSEUDO-CLASES

input:read-write{..}



The image shows a screenshot of a code editor with two panels. The left panel is titled 'HTML' and contains the following code:

```
1 <input type="text" value="Escribe lo que quieras  
   aquí.">  
2
```

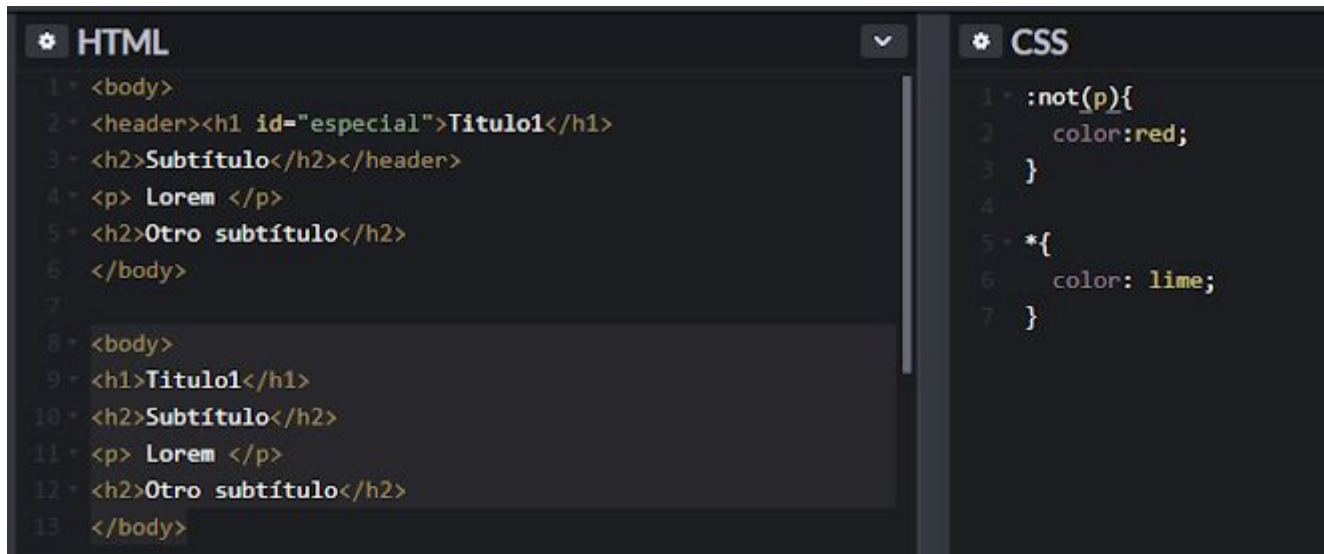
The right panel is titled 'CSS' and contains the following code:

```
1  
2 * input:read-write { background: lightgray; }
```

2. CSS

PSEUDO-SELECTORES: PSEUDO-CLASES

:not(){..}

A screenshot of a code editor with two panels. The left panel is titled 'HTML' and shows two code blocks. The first block (lines 1-7) includes an 'id="especial"' attribute on the first h1. The second block (lines 8-13) is a simplified version without the 'id'. The right panel is titled 'CSS' and shows two CSS rules. The first rule uses ':not(p)' to set the color to red, and the second rule uses '*' to set the color to lime.

```
HTML
1 <body>
2 <header><h1 id="especial">Titulo1</h1>
3 <h2>Subtitulo</h2></header>
4 <p> Lorem </p>
5 <h2>Otro subtitulo</h2>
6 </body>
7
8 <body>
9 <h1>Titulo1</h1>
10 <h2>Subtitulo</h2>
11 <p> Lorem </p>
12 <h2>Otro subtitulo</h2>
13 </body>
```

```
CSS
1 :not(p){
2   color:red;
3 }
4
5 *{
6   color: lime;
7 }
```

2. CSS

PSEUDO-SELECTORES: PSEUDO-CLASES

:not(){.}



```
HTML
1 <body>
2 <header>
3   <h1 id="especial">Titulo1</h1>
4   <h2>Subtítulo</h2></header>
5   <p> Lorem </p>
6   <h2>Otro subtítulo</h2>
7 </body>
8
9 <body>
10 <h1>Titulo1</h1>
11 <h2>Subtítulo</h2>
12 <p> Lorem </p>
13 <h2>Otro subtítulo</h2>
14 </body>

CSS
1 body :not(h1):not(h2){
2   color:red;
3 }
4
5
```

2. CSS

PSEUDO-SELECTORES: PSEUDO-CLASES

<https://drafts.csswg.org/selectors-4/#logical-combination>

Seleccionar múltiples selectores:

:is(){.}

```
:is(div, article) :is(h1, h2, h3, h4, h5, h6) {  
    color: #485e02;  
}
```

2. CSS

PSEUDO-SELECTORES: PSEUDO-CLASES :has()

(selecciona ancestros)

```
/* Selecciona los elementos <a> que contienen un
hijo <img> : */
a:has(> img){
    color: crimson; }
...
<a href="">
    enlace "<a>" con un hijo img
    
</a>
```

enlace "" con un hijo img



```
/* Aplicar estilo a h2 seguido de un hermano h3:
*/
h2:has(+ h3){
    background: cyan; }
.....
<div>
    <h2>Un h2</h2>
    <h3>h3 al que antecede un h2</h3>
</div>
```

Un h2

h3 al que antecede un h2

es un h2

h4 al que antecede un h2

2. CSS

- PSEUDO-CLASES :has()

```
<div>
  <h2>Un h2</h2>
  <h3> es un h3 al que antecede un h2 </h3>
</div>
<div>
  <h2>es un h2</h2>
  <h4>h4 al que antecede un h2</h4>
</div>
<div>
  <h3>es un h3</h3>
  <h4>h4 al que antecede un h3</h4>
</div>
<div>
  <h3>es un h3</h3>
  <h5>h5 al que antecede un h3</h5>
</div>
```

```
/* Aplicar estilo al primer encabezado h2 o
h3 si le sigue un hermano h3 o h4 */
:is(h2, h3):has(+ :is(h3,h4)) {
  background: darkorange;
}
```

Un h2

es un h3 al que antecede un h2

es un h2

h4 al que antecede un h2

es un h3

h4 al que antecede un h3

es un h3

h5 al que antecede un h3

2. CSS

- PSEUDO-CLASES :has()

El siguiente selector coincide con los elementos <section> que no contienen ningún elemento de encabezado:

```
section:not(:has(h1, h2, h3, h4, h5, h6))
```

Hay que tener en cuenta que el orden es importante en el selector anterior. Si se intercambia el anidamiento de las dos pseudoclases, como por ejemplo:

```
section:has(:not(h1, h2, h3, h4, h5, h6))...
```

se obtendría una coincidencia con cualquier elemento <section> que contenga cualquier cosa que no sea un elemento de encabezado.

2. CSS

- PSEUDO-CLASES :has()

```
<div>
  <h3>es un h3</h3>
  <h4>es un h4 </h4>
</div>

<div>
  <h2>Un h2</h2>
  <h3> es un h3 al que antecede un h2 </h3>
</div>

<div>
  <h2>es un h2</h2>
  <h4>h4 al que antecede un h2</h4>
</div>
```

```
/* OR Selecciona el h2 o h3 si es que contiene un h3 o un h4 */
:is(h2, h3):has(+ h3, + h4) {
  background: red;
}

/* AND. Selecciona DIV si tiene h3 y h4 como elementos hijos */
div:has(h3):has(h4){
  background: red;
}
```

2. CSS

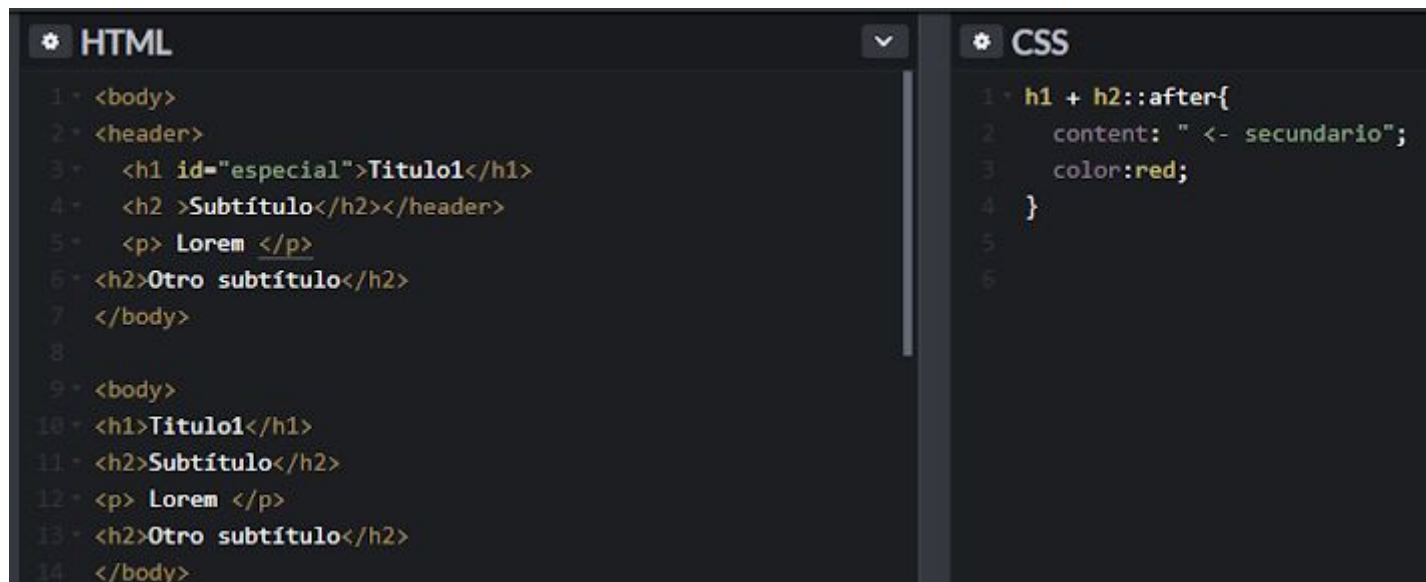
PSEUDO-SELECTORES: PSEUDO-ELEMENTO

Los pseudo-elementos son parecidos a las pseudo-clases, con alguna diferencia. Estos son claves, ahora precedidas por (::), que se añaden al final del selector para elegir cierta parte de un elemento.

- :: en CSS3
- : en CSS2

2. CSS

PSEUDO-SELECTORES: PSEUDO-ELEMENTO ::after



```
HTML
1 <body>
2 <header>
3 <h1 id="especial">Titulo1</h1>
4 <h2>Subtítulo</h2></header>
5 <p> Lorem </p>
6 <h2>Otro subtítulo</h2>
7 </body>
8
9 <body>
10 <h1>Titulo1</h1>
11 <h2>Subtítulo</h2>
12 <p> Lorem </p>
13 <h2>Otro subtítulo</h2>
14 </body>

CSS
1 h1 + h2::after{
2   content: " <- secundario";
3   color:red;
4 }
5
6
```

2. CSS

PSEUDO-SELECTORES: PSEUDO-ELEMENTO

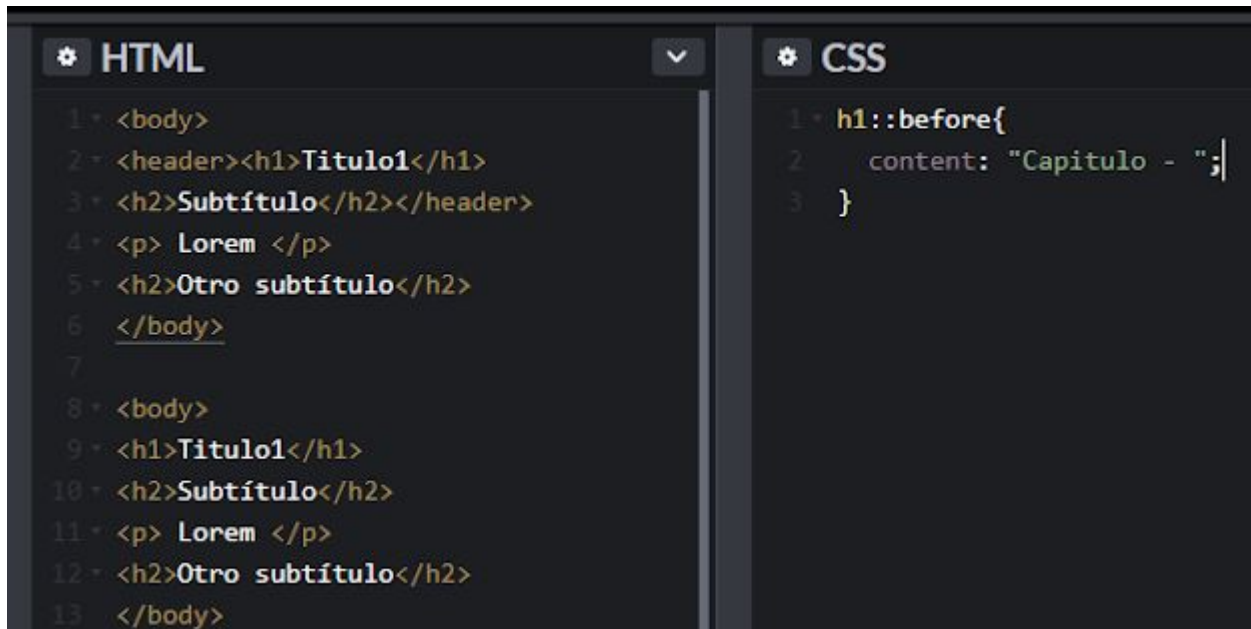
```
HTML
1 <body>
2 <header>
3 <h1 id="especial">Titulo1</h1>
4 <h2>Subtitulo</h2></header>
5 <p> Lorem </p>
6 <h2>Otro subtitulo</h2>
7 </body>
8
9 <body>
10 <h1>Titulo1</h1>
11 <h2>Subtitulo</h2>
12 <p> Lorem </p>
13 <h2>Otro subtitulo</h2>
14 </body>

CSS
1 h1 + h2::after{
2   content:
3   url("data:image/png;base64,iVBORw0KGgoAAAANSUhEUgAAADIAAAAYCAYAAAAeP41xAAA
4   AAXNSR0IArs4c6QAAAAARnQ1BAACxjwv8YQUAAAAJcEhZcwAADsMAAA7DAcdvqGQAAAOFSURBV
5   GhD7dhZqE1RHMfxY55K1JBZMHR1npIpD+QFmR54kCnzAw+mZMgDIUNKHpQHkiSRMo9JyCxtkj
6   hyZB5/P60+7/9771n2udc9+zYv/rUbZ2191777nPW/q8dixI15pT/PhXRHQuwHQdxCDuxEH2gP
7   qFMNczGQ/zK4DnmoSZCFd2B20g26XQeox9CkVH4Aj/BbzidLZ1F6ViBI/iKxL6TUNCxHfYpPT
8   3RjrRGqtTHWvIL/41xKEja4R1sMi/RF9mm657CxuvCQqLccxI2idfogKBphRem41zBZGjF02IwB
9   HDw16IT2M118HLNQ01r5Y/nfcQ2MEeZzy/sRPO855vd8JMP5g3GokxcSGT3wCXyC3Z180xk61gc
10  cxy4cxvYeSTvRaEG1kK/BX9g3faqKIsMhs7jUx0z4RcVLdedEDgtcAf+Asw5IEfaw/8TzyJQ6
11  uEB/OR1+8/jIpahvJK4wGj5zjoq+mzgDyxHLRQqp2DzWa2GbkKv1CZvA1VqFdpzYPNRRZ1V9EC
12  yQddQAZ1S0XX//Jk26q0+QTMUNicVqRkzCLdgg5YgU1SS34T6L1ZDlugz9bmxBUqU/TgtT1dv
13  kQq6015DNbZZFoddoX11yRTxf+DWCZIVEhb2D1qSBYdVINfdTR6+I1Gp1SBDn6AK9UQ4roM/U
14  5A10JEr/nUY3WGiXSChq1+Qs4jGEIry5009D98HPUnBug0L520pNzAsIarVYao813B+Kx0sdMR
15  dgzFzZ1V/xfcymogaSoIYAUZ21tyNDkccveW2Pu4TSC7Dv0KNDexea9CrHrrmKGgJk0Gysdn1
16  LoAo5U9RHff12N90CkSwzYGPjS/Fb19BFDQ6ivbmVTEU/Q03wm1AxtlInlQuLyvMMcJLesPGv1
17  PDZnBRQ8Do4vX1sGN4W1L1dRW/pfXuIpeyvC3sGLqG2BPXoD1BLtESvR6Jr4XSUd91yHV599X
18  wIzXowMQNG9SQR5pBT9/7sGMm0t1Tn6bIJ5thx9ynBpUf1qC9Rr4nsDTEAIwp0h81H15SRJW5d
19  qc27/hzT7fwF3/1fa6NsEZzuwSb7zMUf5/jYR+IaprQvJN10R103HaXer01H57vICrLtoB1/Pw
20  C0bk189s1eGtQKnpaa1C612VhoZ2r9jVpN3u94F+Hhs0J9ETW0SZrG1ThbC0wzUFz+SuvTaNE1
21  RIISpR/JLHybz0r32o3TYA1AAAAAEIFtkSuQmCC");
22 }
```

<https://icons8.com/>

2. CSS

PSEUDO-SELECTORES: PSEUDO-ELEMENTO
::before

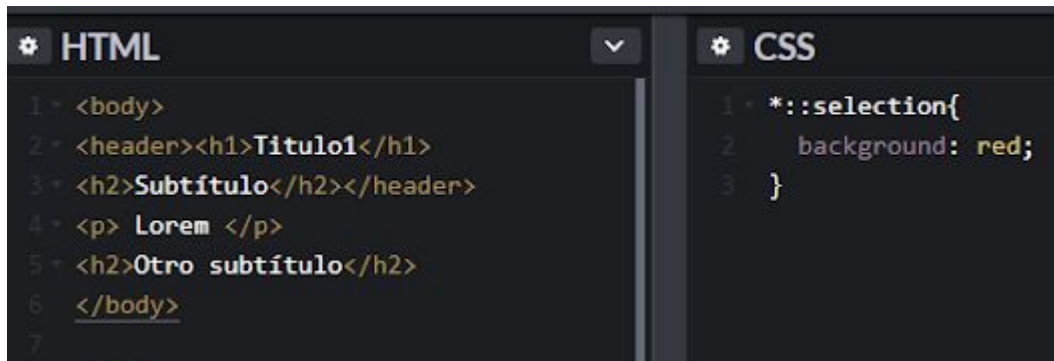


```
HTML
1 <body>
2 <header><h1>Titulo1</h1>
3 <h2>Subtítulo</h2></header>
4 <p> Lorem </p>
5 <h2>Otro subtítulo</h2>
6 </body>
7
8 <body>
9 <h1>Titulo1</h1>
10 <h2>Subtítulo</h2>
11 <p> Lorem </p>
12 <h2>Otro subtítulo</h2>
13 </body>

CSS
1 h1::before{
2   content: "Capítulo - ";
3 }
```

2. CSS

PSEUDO-SELECTORES: PSEUDO-ELEMENTO
::selection



The image shows a code editor with two panels. The left panel is titled 'HTML' and contains the following code:

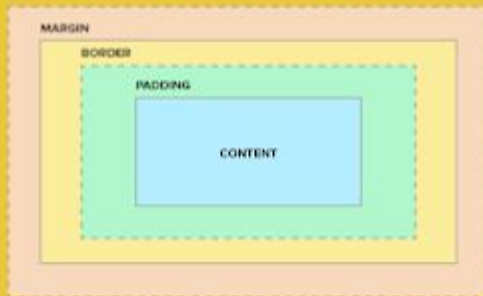
```
1 <body>
2 <header><h1>Titulo1</h1>
3 <h2>Subtítulo</h2></header>
4 <p> Lorem </p>
5 <h2>Otro subtítulo</h2>
6 </body>
7
```

The right panel is titled 'CSS' and contains the following code:

```
1 *::selection{
2   background: red;
3 }
```

modelo de cajas

Box Model Css



2. CSS

MODELO DE CAJAS

Hay dos tipos de cajas que determinan el comportamiento de la caja:

- Cajas de bloque
- Cajas inline

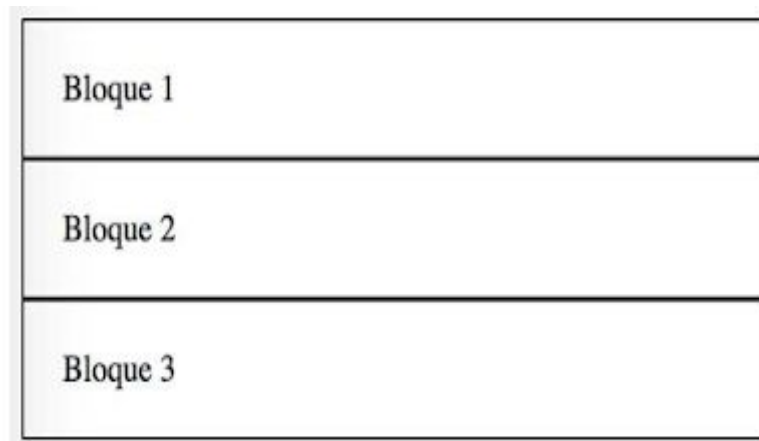
❖ Inline-block

Cada elemento HTML es por defecto de un tipo u otro, pero también se puede cambiar mediante CSS.

2. CSS

CAJAS DE BLOQUE

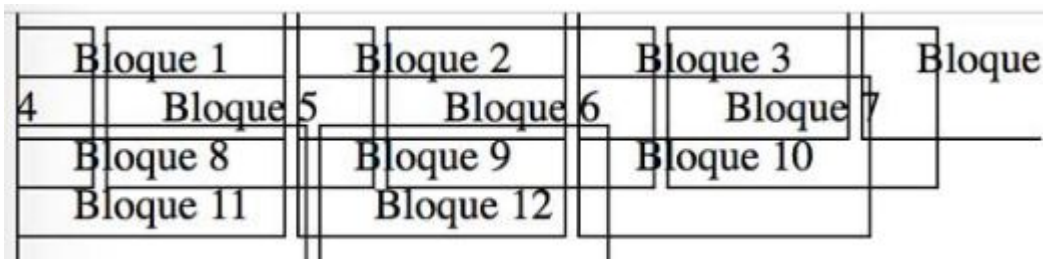
- Crean nuevos bloques de contenido.
- Son renderizadas verticalmente siguiendo el orden en el código.
- Se expanden para ocupar todo el ancho del elemento contenedor (salvo tablas)
- Es lo que se conoce como flujo normal de la página



2. CSS

CAJAS DE INLINE

- No forman nuevos bloques de contenido
- Ocupan el ancho que necesiten para su contenido.
- No excedan el ancho del contenedor, si es necesario pasan a ocupar varias líneas



2. CSS

CAJAS DE INLINE- BLOCK

- Los elementos inline-block son mezcla de los dos, se colocan en línea pero mantienen las propiedades width y height.
- Una práctica común es convertir elementos bloque en elementos inline-block para maquetar un espacio de una web.

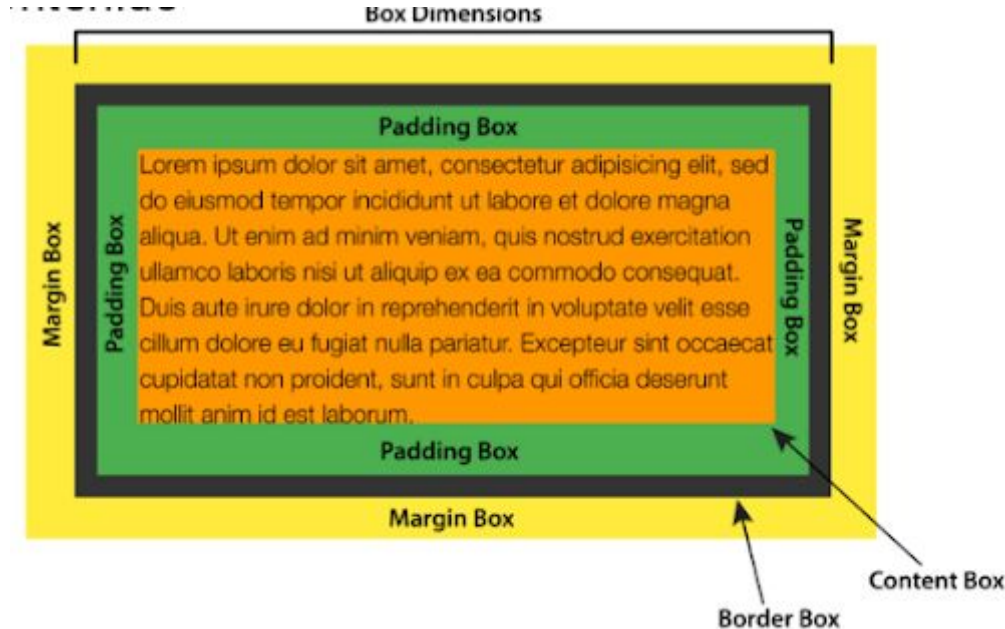
Bloque 1	Bloque 2	Bloque 3
----------	----------	----------

2. CSS

MODELO DE CAJA

En cada caja podemos distinguir varios elementos que son los que determinan su tamaño final:

- Área de contenido
- Padding
- Borde
- Margen



2. CSS

MODELO DE CAJA ->

PROPIEDAD

width

sintaxis CSS

```
width: auto|value|initial|inherit;
```

Valores de propiedad

Value	Description
auto	Default value. The browser calculates the width
<i>length</i>	Defines the width in px, cm, etc.
%	Defines the width in percent of the containing block
initial	Sets this property to its default value. Read about <i>initial</i>
inherit	Inherits this property from its parent element. Read about <i>inherit</i>

2. CSS

MODELO DE CAJA ->

PROPIEDAD

height

sintaxis CSS

```
height: auto|length|initial|inherit;
```

Valores de propiedad

Value	Description
auto	The browser calculates the height. This is default
<i>length</i>	Defines the height in px, cm, etc.
%	Defines the height in percent of the containing block
initial	Sets this property to its default value. Read about <i>initial</i>
inherit	Inherits this property from its parent element. Read about <i>inherit</i>

2. CSS

MODELO DE CAJA -> PROPIEDADES

width y height

Estas propiedades no se aplican a todos los elementos, solamente a:

- Todos los elementos de bloque
- Imágenes
- Filas y grupos de fila de tabla (width)
- Columnas y grupos de columna de tabla (height)

2. CSS

MODELO DE CAJA ->

PROPIEDADES

**Min-width y
min-height**

```
min-width: length|initial|inherit;
```

```
min-height: length|initial|inherit;
```

Value	Description
<i>length</i>	Default value is 0. Defines the minimum width in px, cm, etc.
%	Defines the minimum width in percent of the containing block
initial	Sets this property to its default value. Read about <i>initial</i>
inherit	Inherits this property from its parent element. Read about <i>inherit</i>

2. CSS

MODELO DE CAJA ->

PROPIEDADES

margin

sintaxis CSS

```
margin: length|auto|initial|inherit;
```

Valores de propiedad

Value	Description
<i>length</i>	Specifies a margin in px, pt, cm, etc. Default value is 0
%	Specifies a margin in percent of the width of the containing element
auto	The browser calculates a margin
initial	Sets this property to its default value. Read about <i>initial</i>
inherit	Inherits this property from its parent element. Read about <i>inherit</i>

2. CSS

MODELO DE CAJA -> PROPIEDADES

margin

- 1 valor:
 - Se aplica a los 4 márgenes
- 2 valores:
 - Primero: margen superior e inferior
 - Segundo: margen izquierdo y derecho
- 3 valores:
 - Primero: margen superior
 - Segundo: márgenes izquierdo y derecho
 - Tercero: margen inferior
- 4 valores:
 - El orden es: superior, derecho, inferior e izquierdo

2. CSS

MODELO DE CAJA -> PROPIEDADES

margin

- También se pueden indicar individualmente

```
margin-top: length|auto|initial|inherit;
```

```
margin-right: length|auto|initial|inherit;
```

```
margin-bottom: length|auto|initial|inherit;
```

```
margin-left: length|auto|initial|inherit;
```

2. CSS

MODELO DE CAJA -> PROPIEDADES

margin

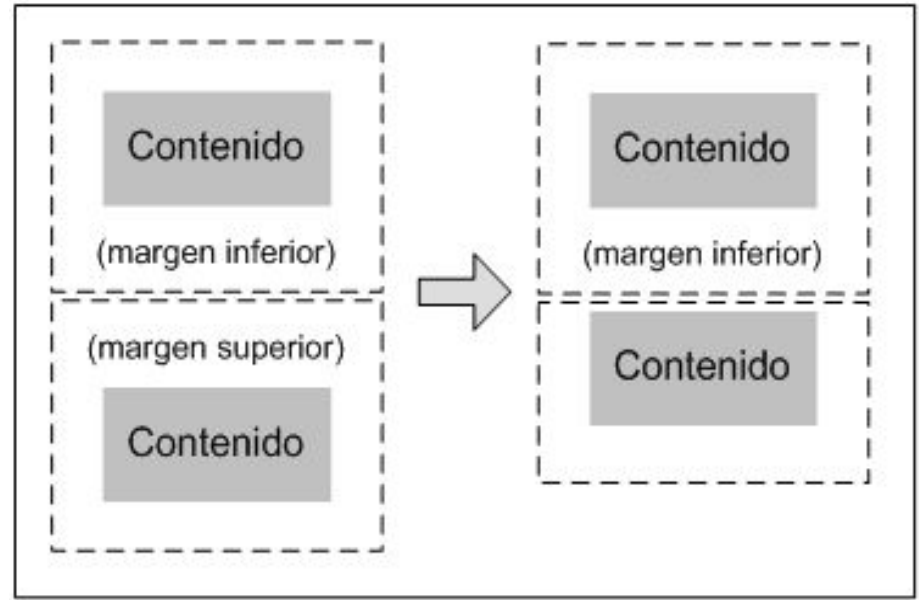
- Los **márgenes superior e inferior** solo se pueden aplicar a elementos de bloque.
- Los **márgenes izquierdo y derecho** se pueden aplicar a cualquier elemento.

2. CSS

MODELO DE CAJA -> PROPIEDADES

margin

- El comportamiento de los márgenes verticales es más complejo
- Cuando se juntan dos o más márgenes verticales se fusionan y la altura del nuevo margen será la del mayor de ambos márgenes. Nota: entre **html** y **body** al estar anidados no se produce solapamiento y no se fusionan.

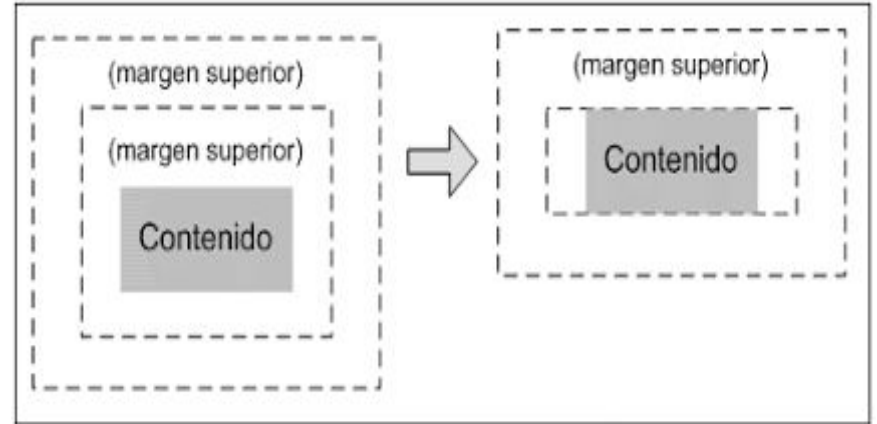


2. CSS

MODELO DE CAJA -> PROPIEDADES

margin

- Si un elemento está contenido dentro de otro, sus márgenes verticales se fusionan y resultan en un margen de la misma altura que el mayor margen de los que se han fusionado.



2. CSS

MODELO DE CAJA -> PROPIEDADES **margin**

márgenes colapsados

Las márgenes verticales (superior -top- e inferior -bottom-) de los elementos bloque (block) se colapsan o combinan en ciertas ocasiones, y predomina la margen más grande, o alguna de ellas, en caso que sean iguales.

Esto ocurre en tres casos principalmente:

- **Hermanos adyacentes:** Los márgenes de los hermanos adyacentes son colapsados (excepto cuando el último hermano necesita ser limpiado luego de los flotantes).
- **Padre y primer/último hijo:** Si no hay un borde, padding, contenido en línea, o limpiado para separar el margin-top de un bloque del margin-top de su primer bloque hijo, o no hay borde, padding, contenido en línea, height, min-height, o max-height para separar el margin-bottom de un bloque del margin-bottom de su último hijo, entonces esos márgenes colapsan. El margen colapsado termina fuera del padre.
- **Bloques vacíos:** Si no hay borde, padding, contenido en línea, height, o min-height para separar el margin-top de un bloque de su margin-bottom, entonces sus márgenes superiores e inferiores colapsan.

2. CSS

MODELO DE CAJA -> PROPIEDADES **margin**

márgenes colapsados

En el ejemplo a nuestra derecha se aplica el segundo caso "Padre y primer/último hijo". Aplicando un borde transparente de 1px para demostrar lo que dice la documentación: *"Si no hay borde o ... que separe el margin-top padre del margin-top hijo, ..."*, entonces al agregar el borde, básicamente no se colapsan.

```
body{
  margin: 0 auto;
  padding: 0;
}
#padre{
  width: 400px; height: 400px;
  background: orange;
  border-top: solid 1px transparent;
}
#hijo {
  width: 100px; height: 100px;
  background: green; margin: 20px;
}
.....
<section id="padre">
  <div id="hijo" >1</div>
</section>
```

2. CSS

MODELO DE CAJA -> PROPIEDADES **margin**

márgenes colapsados

SI se combinan *márgenes positivos y negativos* se suman.

En los elementos vacíos también hay solapamiento.

Se quita `border`, `padding` o `display:inline-block`

2. CSS

MODELO DE CAJA

-> PROPIEDADES

padding

sintaxis CSS

```
padding: length|initial|inherit;
```

Valores de propiedad

Value	Description
<i>length</i>	Specifies the padding in px, pt, cm, etc. Default value is 0
%	Specifies the padding in percent of the width of the containing element
initial	Sets this property to its default value. Read about <i>initial</i>
inherit	Inherits this property from its parent element. Read about <i>inherit</i>

2. CSS

MODELO DE CAJA -> PROPIEDADES

padding

Al igual que en el caso de **margin** admite entre 1 y 4 valores.

E igualmente se puede indicar por separado cada uno de los lados.

```
padding-left: length|initial|inherit;
```

```
padding-bottom: length|initial|inherit;
```

```
padding-top: length|initial|inherit;
```

```
padding-right: length|initial|inherit;
```

2. CSS

MODELO DE CAJA -> PROPIEDADES

border sintaxis CSS

```
border: border-width border-style border-color|initial|inherit;
```

Valores de propiedad

Value	Description
<u>border-width</u>	Specifies the width of the border. Default value is "medium"
<u>border-style</u>	Specifies the style of the border. Default value is "none"
<u>border-color</u>	Specifies the color of the border. Default value is the color of the element
<u>initial</u>	Sets this property to its default value. Read about <i>initial</i>
<u>border-radius</u>	Sets all the four border-*-radius properties for rounded corners

2. CSS

MODELO DE CAJA -> PROPIEDADES

border-style

- **dotted** - Define un borde punteado.
- **dashed** - Define un borde discontinuo.
- **solid** - Define un borde sólido.
- **double** - Define un doble borde.
- **groove** - Define un borde ranurado en 3D. El efecto depende del valor del color del borde.
- **ridge** - Define un borde acanalado en 3D. El efecto depende del valor del color del borde.
- **inset** - Define un borde de inserción 3D. El efecto depende del valor del color del borde.
- **outset** - Define un borde de inicio 3D. El efecto depende del valor del color del borde.
- **none** - No define fronteras
- **hidden** - Define un borde oculto.

2. CSS

MODELO DE CAJA -> PROPIEDADES

border-style

HTML

```
1 <div class="uno">1</div><br>
2 <div class="dos">2</div><br>
3 <div class="tres">3</div><br>
4
5 <div class="cuatro">1</div><br>
6 <div class="cinco">2</div><br>
7 <div class="seis">3</div><br>
```

CSS

```
1 .uno { border: 5px solid blue; border-bottom-color: red; }
2 .dos { border: 5px red; border-style: solid double; }
3 .tres { border: double; border-color: red blue green black; }
4 .cuatro { border: dotted; }
5 .cinco { border-top: dashed; border-left:dotted; border-
  right:double; border-bottom:none; }
6 .seis { border-top: 10px groove; border-left: 10px ridge;
  border-right: 10px inset; border-bottom: 10px outset; }
```

2. CSS

MODELO DE CAJA -> PROPIEDADES

background

Property Values

Value	Description	CSS
<u>background-color</u>	Specifies the background color to be used	1
<u>background-image</u>	Specifies ONE or MORE background images to be used	1
<u>background-position</u>	Specifies the position of the background images	1
<u>background-size</u>	Specifies the size of the background images	3
<u>background-repeat</u>	Specifies how to repeat the background images	1
<u>background-origin</u>	Specifies the positioning area of the background images	3
<u>background-clip</u>	Specifies the painting area of the background images	3
<u>background-attachment</u>	Specifies whether the background images are fixed or scrolls with the rest of the page	1
initial	Sets this property to its default value. <u>Read about initial</u>	3
inherit	Inherits this property from its parent element. <u>Read about inherit</u>	2

2. CSS

MODELO DE CAJA -> PROPIEDADES

background-color

- indica el color de fondo.
- Se aplica tanto al contenido como al padding.

```
background-color: color|transparent|initial|inherit;
```

Valores de propiedad

Value	Description
<i>color</i>	Specifies the background color. Look at CSS Color Values for a complete list of possible color values.
<i>transparent</i>	Specifies that the background color should be transparent. This is default
<i>initial</i>	Sets this property to its default value. Read about <i>initial</i>
<i>inherit</i>	Inherits this property from its parent element. Read about <i>inherit</i>

2. CSS

MODELO DE CAJA -> PROPIEDADES

background-image

Propiedad	background-image
Valores	url none inherit

```
body { background-image: url("imagenes/fondo.png") }
```

- La imagen se muestra delante del color.
- Las imágenes se indican a través de su url

2. CSS

MODELO DE CAJA -> PROPIEDADES

background-image

- Si la imagen es más grande que el contenedor se recorta.
- Si es más pequeña se repite tanto horizontal como verticalmente.



2. CSS

MODELO DE CAJA -> PROPIEDADES

background-repeat

- Con esta propiedad controlamos en qué eje se repite y en cuál no.

Propiedad	background-repeat
Valores	repeat repeat-x repeat-y no-repeat inherit

2. CSS

MODELO DE CAJA -> PROPIEDADES

background-position

Propiedad	background-position
Valores	<code>((porcentaje unidad de medida left center right) (porcentaje unidad de medida top center bottom)?) ((left center right) (top center bottom)) inherit</code>

- Si no repetimos la imagen podemos escoger la posición de la misma dentro del contenedor.
- Valor por defecto: 0% 0%
- El valor de porcentaje es poco intuitivo: si se indica (x%, y%) el navegador coloca el punto (x%, y%) de la imagen en el punto (x% y%) del elemento.

2. CSS

MODELO DE CAJA -> PROPIEDADES

background-position

- Las palabras claves son:
 - top: 0%
 - left: 0%
 - center: 50%
 - bottom: 100%
 - right: 100%
- Se pueden combinar palabras clave con unidades.
- Si solo se utilizan palabras clave el orden es indiferente.

2. CSS

MODELO DE CAJA -> PROPIEDADES

¿Cuál es el ancho real de un elemento?

El ancho que ocupa una caja está determinado por:

- el ancho del contenido
- más el tamaño del padding
- más el tamaño de la línea de borde

2. CSS

MODELO DE CAJA -> PROPIEDADES

```
<div class="uno">uno</div>  
<div class="dos">dos</div>
```

```
.uno{  
    width : 1000px;  
    height : 100px;  
    background-image: url("logos.png") ;  
    background-position: 75%;  
    background-repeat: no-repeat;  
    border: solid;  
}  
.dos{  
    width : 1000px;  
    height : 100px;  
    background: url("logos.png") no-repeat 75%;  
    border: solid;  
}
```

2. CSS

MODELO DE CAJA -> PROPIEDADES

Box-sizing

sintaxis CSS

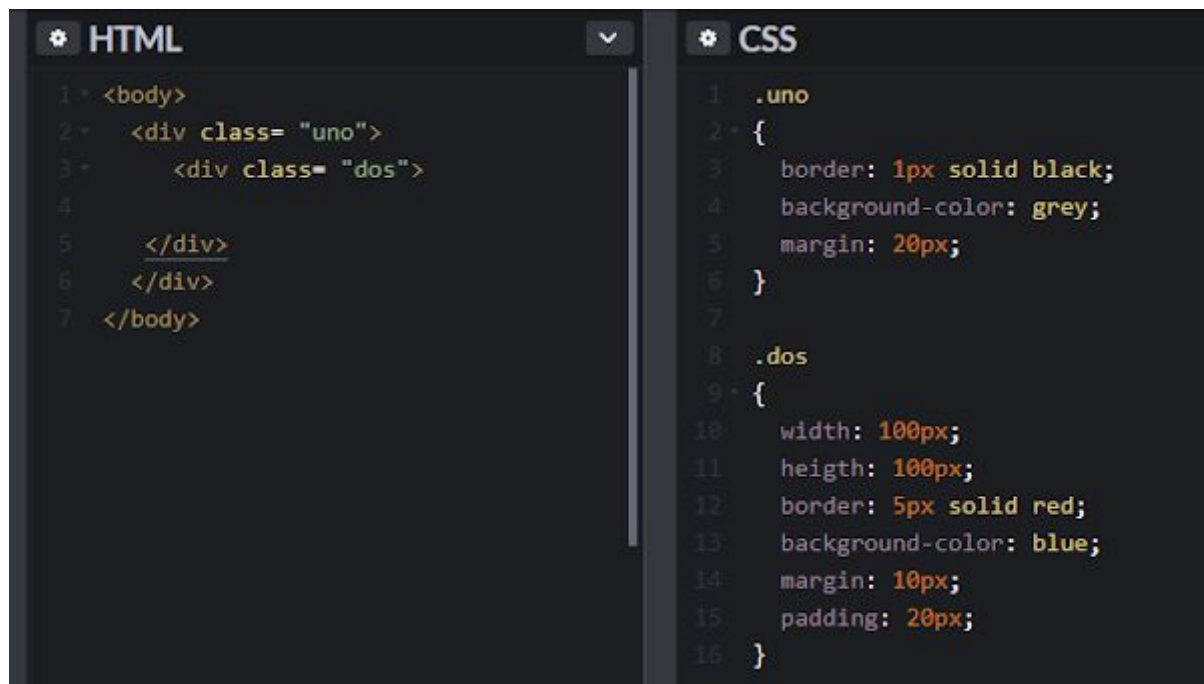
```
box-sizing: content-box|border-box|initial|inherit;
```

Valores de propiedad

Value	Description
content-box	Default. The width and height properties (and min/max properties) includes only the content. Border, padding, or margin are not included
border-box	The width and height properties (and min/max properties) includes content, padding and border, but not the margin
initial	Sets this property to its default value. Read about <i>initial</i>
inherit	Inherits this property from its parent element. Read about <i>inherit</i>

2. CSS

MODELO DE CAJA -> PROPIEDADES



2. CSS

MODELO DE CAJA -> Unidades de longitud absoluta

En general, siempre tienen el mismo tamaño.

Unidad	Nombre	Equivale a
cm	Centímetros	1cm = 96px/2,54
mm	Milímetros	1mm = 1/10 de 1cm
Q	Cuartos de milímetros	1Q = 1/40 de 1cm
in	Pulgadas	1in = 2,54cm = 96px
pc	Picas	1pc = 1/6 de 1in
pt	Puntos	1pt = 1/72 de 1in
px	Píxeles	1px = 1/96 de 1in

2. CSS

MODELO DE CAJA -> Unidades de longitud relativa

En general, son relativas a algo más, por ejemplo, al tamaño de letra del elemento o al tamaño de la ventana gráfica.

Unidad	Relativa a
<code>em</code>	Tamaño de letra del elemento padre, en el caso de propiedades tipográficas como <code>font-size</code> , y tamaño de la fuente del propio elemento en el caso de otras propiedades, como <code>width</code> .
<code>ex</code>	Altura x de la fuente del elemento.
<code>ch</code>	La medida de avance (ancho) del glifo "0" de la letra del elemento.
<code>rem</code>	Tamaño de la letra del elemento raíz.
<code>lh</code>	Altura de la línea del elemento.
<code>vw</code>	1% del ancho de la ventana gráfica(<i>viewport</i>)
<code>vh</code>	1% de la altura de la ventana gráfica(<i>viewport</i>)
<code>vmin</code>	1% de la dimensión más pequeña de la ventana gráfica(<i>viewport</i>).
<code>vmax</code>	1% de la dimensión más grande de la ventana gráfica (<i>viewport</i>).

2. CSS

lvh, svh y dvh, las unidades de medida para pantallas móviles

En los navegadores móviles el ancho, y sobre todo el alto del viewport, varían dinámicamente en función de la aparición y desaparición de ciertos elementos del navegador, como la barra de direcciones, los botones de navegación, etc... que aparecen y desaparecen al hacer scroll en cierta dirección y otras circunstancias.

Para evitar este problema y poder **adaptarnos en cada momento al verdadero tamaño del viewport**, se han definido dos tamaños de *viewport* en el caso de los móviles: **Viewport Grande** y **Viewport Pequeño** y por ello, se incluyen unas nuevas unidades que complementan a las que ya conocemos, y que llevan el prefijo lv (large viewport) y sv (small viewport) para distinguirlas:

Unidades para dimensiones constantes del viewport:

Cuando no hay elementos del navegador interfiriendo tenemos: **lvw, lvh, lvmin, lvmax, lvi y lvb**: que indican respectivamente el ancho, alto, mínimo, máximo y dimensiones del eje inline y del eje block en el "viewport grande".

Cuando los elementos del navegador móvil sí están visibles: **svw, svh, svmin, svmax, svi y svb** en el "viewport pequeño":

Unidades para dimensiones dinámicas del viewport:

¿Qué pasa si no queremos ceñirnos a uno u otro tamaño, sino al tamaño que haya en cada momento en función de si se ven o no los elementos? La especificación nos ayuda gracias a las unidades dinámicas del viewport, que son equivalentes a las que acabamos de ver pero con el prefijo **dv** delante (de "dynamic viewport"): **dvw, dvh, dvi, dvb, dvmin y dvmax**.

En este caso los valores de estas unidades coinciden con las del área visible de la página. Es decir, si los elementos del navegador están ocultos coinciden con las dimensiones del viewport grande. Y si los elementos del navegador se muestran, entonces coinciden con los valores de las unidades del viewport pequeño.